

jcode Swarm Handbook

A short, source-verified tour of how jcode runs many AI agents in parallel — with idiomatic TypeScript equivalents for every Rust pattern.

Who this is for

You don't need to read Rust. Every pattern in this handbook is explained in plain English first, shown as real Rust evidence from the jcode codebase second, and taught through a hand-written idiomatic TypeScript equivalent third. The TypeScript is not a literal port — jcode has no TypeScript implementation of its swarm internals — it's a clearly-labeled translation of the same design pattern into a language you can actually run and modify.

How this handbook was built (and why you can trust its citations)

This handbook was generated by a 4-phase pipeline of AI agents working against the live jcode source tree (commit 5ae238574, 2026-08-12):

1. **Explore** — one agent mapped every file, struct, and function this handbook cites, recording exact `file:line` locations.
2. **Analyze** — four agents turned that map into per-chapter technical notes, independently re-opening every cited file rather than trusting the map's paraphrase, and explicitly flagging anything they couldn't verify as an open question instead of guessing.
3. **Write** — four agents turned those notes into the tutorial chapters below, preserving every citation and every flagged uncertainty.
4. **Review & Revise** — two independent agents, who did not write any of the drafts, adversarially re-checked every single citation and TypeScript snippet against the actual source a second time. Where they found miscited line ranges or an undisclosed caveat, dedicated revision agents fixed exactly those issues — no broader rewrites.

Every Rust code block below carries an inline `// jcode: file:line` citation you can check yourself. Every TypeScript block is explicitly labeled as an idiomatic equivalent, not real jcode code. Where the pipeline could not verify a claim, the chapter says so honestly rather than smoothing it into confident prose — you'll see a few of these; they're a feature of the process, not a gap in it.

Table of contents

1. [Why Parallel Agents, and Why This Codebase](#)
2. [Anatomy of a Spawn](#)
3. [Fan-out / Fan-in Patterns](#)
4. [Structured Concurrency & Cancellation](#)
5. [Shared State Without Locks-as-a-Crutch](#)
6. [The Task DAG \(Deep Mode\)](#)
7. [Communication Topology](#)
8. [Design-Pattern Glossary & Further Reading](#)

Read time: roughly 1.5–2.5 hours end to end (~22,500 words) — a train ride, or an easy afternoon. Chapters 1–2 give you the vocabulary and the spawn mechanism; 3–5 cover the core concurrency patterns; 6–7 cover the task-graph and communication layers; 8 is a keep-open-in-a-tab reference.

Chapter 1 — Why Parallel Agents, and Why This Codebase

One agent, one thread, one bottleneck

A coding agent, at its simplest, is a loop: read something, think about it, take one action, look at what happened, repeat. One tool call at a time. One working directory. One conversation thread carrying all the context.

That loop is a fine model for a scoped task — "fix this one function," "explain this error." It falls apart the moment the work is wide instead of deep. "Explore this entire codebase." "Fix these forty independent lint violations." "Implement six unrelated features before end of day." None of those sub-tasks depend on each other, and yet a single serial agent will still do them one after another, paying the full cost of each one before starting the next.

This is the same realization that gave us parallel CI: if your test suite takes an hour and none of the tests depend on each other, you don't make the suite faster by writing quicker tests — you shard it across workers and run it in twenty minutes. The jcode swarm system applies that idea to coding agents. The hard part was never "how do I start more than one agent" — `tokio::spawn` on its own can do that in one line. The hard part is: how do you let one agent become many, all working at once, without turning shared state into a race condition and without losing track of who is doing what for whom.

This chapter builds the vocabulary you'll need for the rest of the book, and is honest about one thing up front: not every term you'll see in jcode's own design docs corresponds to a real, load-bearing piece of code. Part of learning this system is learning to tell the difference.

Why jcode is a good codebase to learn this from

jcode's swarm layer is not built on an actor framework, a job queue, or any kind of distributed-systems library. There's no `actix`, no `ractor`, no message broker. It's hand-rolled directly on top of Tokio's own primitives — `JoinSet`,

CancellationToken, broadcast channels, RwLock, Mutex, and plain `tokio::spawn`. This was confirmed by reading the import lists at the top of the two files that do most of the orchestration work:

```
// jcode: crates/jcode-app-core/src/server/swarm.rs:1-20
use futures::future::try_join_all; // line 9
use tokio::sync::{Mutex, RwLock, broadcast}; // line 20
```

No actor-model crate appears anywhere in the swarm code path. That's good news for a reader who doesn't know Rust: every concurrency primitive jcode uses here is a *named, ordinary* building block, not framework-specific magic. Each one has a direct, nameable equivalent in TypeScript/Node — `Promise.all`, `worker_threads`, `EventEmitter`, `AbortController` — which is exactly the mapping this handbook is going to draw, chapter by chapter.

Vocabulary

Session, and Agent

A **session** is the persistent identity and conversation state of one running agent — think "the row in the database that says this conversation exists and here's its history." An **agent** is the live, in-memory object that actually drives the loop — it holds the session and does the work. Spawning a new member of a swarm means creating a brand-new session/agent pair, literally:

```
// jcode: crates/jcode-app-core/src/server/swarm.rs:1548-1551
let mut session = Session::create(
    Some(session_id),
    Some(format!("{}", (@{} swarm)", description, subagent_type)),
);
```

(This particular snippet lives inside a debug-only code path — more on that caveat in Chapter 2 — but the `Session::create` call shape is representative of what "creating a session" looks like everywhere in the codebase.)

`Session::create` just returns an ordinary struct value — nothing registers itself anywhere by calling this. The caller decides what to do with it next.

```
// idiomatic TS equivalent – this code does not exist in jcode
class Session {
    static create(id: string | null, title: string | null): Session {
        return new Session(id ?? crypto.randomUUID(), title);
    }
    private constructor(public id: string, public title: string | null) {}
}

// Calling Session.create() does nothing but hand you a value –
// exactly like calling `new Session(...)` directly. No registry,
// no side effects, until *you* do something with it.
const session = Session.create(null, "api reviewer (@agent swarm)");
```

There's a third term worth knowing now, even though its full mechanics belong to Chapter 2: a **SwarmMember** is the live bookkeeping record for "a session that is part of a swarm." It's a separate struct from `Session` — tracking things like status, which swarm it belongs to, and who it reports to — constructed at the point a spawn actually registers:

```
// jcode: crates/jcode-app-core/src/server/comm_session.rs:502-528
// (register_visible_spawned_member – constructs the SwarmMember record)
```

So, three related but distinct things: **agent** is the live object doing the work; **session** is its persisted identity; **SwarmMember** is the swarm's own bookkeeping entry about that session. jcode's design docs sometimes use "agent" loosely to mean all three — worth untangling early so later chapters don't feel sloppy about it.

Coordinator

A swarm can have a **coordinator** — a role, not a special kind of agent. It's literally a string, and the tool schema that lets an agent claim it only accepts two values:

```
// jcode: crates/jcode-app-core/src/tool/communicate.rs:2005-2008
"role": { "type": "string", "enum": ["agent", "coordinator"] }
```

At most one member per swarm holds the coordinator role at a time, and its job is narrow: it's the one member allowed to own the swarm's single shared task plan (propose it, approve it, hand out assignments). Only a **root** session — one that wasn't itself spawned by anyone — is allowed to claim the slot, and only

when it's empty or the current holder looks stale (every one of its event channels closed, even without a clean shutdown — a deliberate guard against a dead coordinator wedging the slot forever).

It's worth stressing what "coordinator" is *not*: it isn't "the agent that owns this piece of work" or "the parent of this subtree." That relationship is tracked separately (see `report_back_to_session_id`, below). A member can be a plain "agent" and still be the parent of five other agents; the coordinator role only ever concerns who owns the plan.

```
// idiomatic TS equivalent – this code does not exist in jcode
type Role = "agent" | "coordinator";

interface SwarmMember {
  sessionId: string;
  role: Role;
  isRoot: boolean; // no report_back_to_session_id
}

function tryClaimCoordinator(
  swarmId: string,
  candidate: SwarmMember,
  coordinators: Map<string, string>, // swarmId -> sessionId
  members: Map<string, SwarmMember>,
): boolean {
  if (!candidate.isRoot) return false;
  const current = coordinators.get(swarmId);
  const currentIsStale = current ? isStale(members.get(current)) : true;
  if (current && !currentIsStale) return false;
  coordinators.set(swarmId, candidate.sessionId);
  return true;
}
```

Swarm

A **swarm** isn't a struct with its own lifecycle at all — it's just a `swarm_id` string that a set of `SwarmMembers` happen to share. The grouping itself is one more index sitting in shared state: `swarm_id -> set of member session ids`.

Where does that id come from? Normally it's derived from the repo's git common directory, so that multiple git worktree checkouts of the same repository land in the same swarm:

```
// jcode: crates/jcode-app-core/src/server/util.rs:329-341
pub(crate) fn swarm_id_for_dir(dir: Option<PathBuf>) -> Option<String> {
    if let Ok(sw_id) = std::env::var("JCODE_SWARM_ID") { /* explicit
        override */ }
    let dir = dir?;
    if let Some(git_common) = git_common_dir_for(&dir) {
        return Some(git_common.to_string_lossy().to_string());
    }
    Some(dir.to_string_lossy().to_string())
}
```

There's a nuance, though: root sessions actually default to *owning their own swarm*, rather than automatically sharing one with every other session opened in the same repo. The code's own comment explains why — deriving the id purely from the directory meant every session opened in one repository shared one plan, "even when those sessions were unrelated." JCODE_SWARM_ID is the explicit opt-in for when you *do* want independently-started root sessions to share a swarm on purpose.

Worktree Manager — a term that isn't backed by code

This one is worth calling out plainly, because it's a place where jcode's own design prose gets ahead of its implementation, and a handbook that glossed over it would be teaching something false.

docs/SWARM_ARCHITECTURE.md describes a "Worktree Manager" role: an agent that "owns a single worktree scope" and is "responsible for integration when that worktree scope is done." It reads like a peer of "coordinator."

A full search of the source tree for WorktreeManager or worktree_manager finds no struct, no enum variant, no permission check — nothing. The phrase appears in exactly one place in actual source code: an error message.

```
// jcode: crates/jcode-app-core/src/server/comm_sync.rs:346-353
message: "Only the coordinator, worktree manager, or the target session
    may read full context. Use summary for lightweight
    access.".to_string(),
```

But the function that message guards checks only two conditions — nothing resembling a worktree-manager role:

```
// jcode: crates/jcode-app-core/src/server/comm_sync.rs:178-192
async fn can_read_full_context(...) -> bool {
    if req_session_id == target_session { return true; }
    let members = swarm_members.read().await;
    members.get(req_session_id).map(|member| member.role ==
        "coordinator").unwrap_or(false)
}
```

Self, or coordinator. That's it. Since role is restricted by the tool schema to exactly {"agent", "coordinator"}, there is no third value a session could even hold to represent "worktree manager." The error text is aspirational — leftover language from the design intent — not a real branch in the authorization logic.

Fittingly, jcode's newer design doc (docs/SWARM_TASK_GRAPH.md) states the project's own direction plainly: coordinator and worktree-manager roles are meant to "demote to scheduler policy, not user-facing concepts" going forward. So treat "Worktree Manager" as a vocabulary term describing an *informal* responsibility — an agent can be told, via its spawn prompt, to own integration for one worktree — not a permission the system enforces.

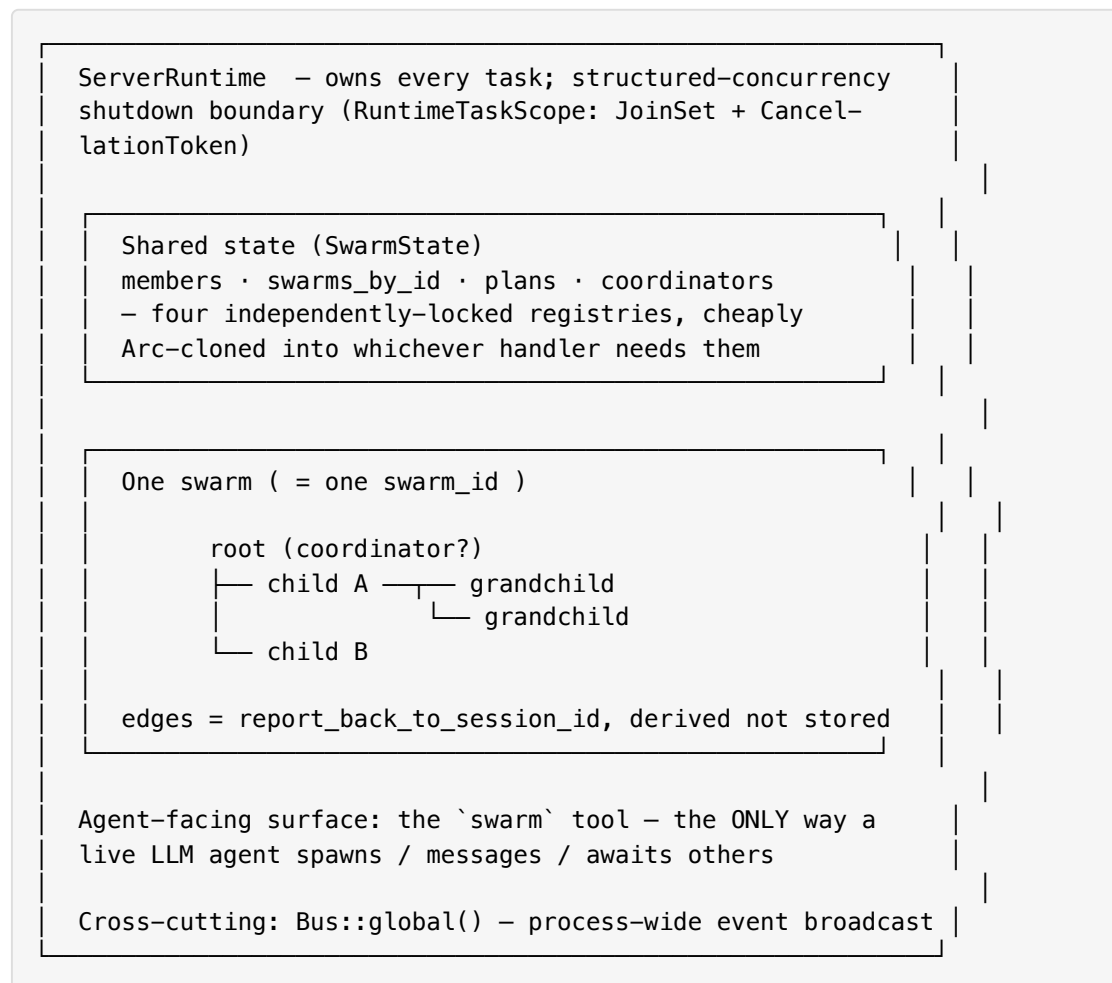
The spawn tree, previewed

Every spawned member carries one field, `report_back_to_session_id`, pointing at whichever session spawned it. That single edge is enough to reconstruct the entire ancestry chain on demand:

```
// jcode: crates/jcode-app-core/src/server/swarm.rs:34-40 (doc comment)
/// The spawner/parent edge is encoded by `report_back_to_session_id`: a
///   child
/// spawned by `P` reports back to `P`. Walking that chain reconstructs
///   the spawn
/// tree without persisting a separate parent field. Cycles (which should
///   never
/// happen) are guarded against with a visited set.
```

The design choice worth noticing here, before Chapter 2 gets into the mechanics: jcode doesn't maintain a tree data structure that could drift out of sync with reality. It stores the minimal edge — "who do I report to" — on each member, and derives the tree by walking it. One field, always consistent by construction, versus a redundant structure that has to be kept in sync by hand.

The architecture, in one picture



The whole thing maps onto TypeScript surprisingly cleanly: one Node process, N concurrently-running async jobs (or, for one specific spawn mode covered in Chapter 2, literal worker_threads) registered in a parent-side `Map<id, WorkerHandle>`, with a single shared event emitter for anything cross-cutting. That's the mental model to carry forward.

A caveat before you go further

If you go reading `swarm.rs` yourself, you'll run into a function called `run_swarm_task` built around `try_join_all` – a classic planner-fans-out-then-awaits-everything shape. It's tempting to assume that's "what happens when an agent calls the swarm tool to spawn workers."

It isn't. Tracing its callers shows both of them are debug-socket commands (`swarm_message:`, `swarm_message_async:`), not anything reachable from a live agent's tool calls. The swarm tool's actual spawn action goes through a completely different function, `spawn_swarm_agent`, which is the real subject of Chapter 2. Keep that distinction in mind — it matters for every chapter that follows.

Next: Chapter 2 — Anatomy of a Spawn walks through exactly what happens, function by function, when a live agent calls `swarm(action: "spawn")`.

Chapter 2 — Anatomy of a Spawn

Two paths, and only one of them is real

Before we trace anything, a correction that matters more than it might seem. Open `swarm.rs` and you'll find a function pair, `run_swarm_task / run_swarm_message`, built around `try_join_all` — the kind of planner-fans-out-and-awaits-everything shape that looks exactly like "the" spawn mechanism a design doc would draw a diagram of.

It is real, working code. It is also **not what happens when a live agent calls the swarm tool**. Tracing every caller of `run_swarm_message` turns up exactly two, and both are debug-socket string commands, not tool calls:

```
// jcode: crates/jcode-app-core/src/server/debug_command_exec.rs:132-139
if trimmed.starts_with("swarm_message:") {
    let msg =
        trimmed.strip_prefix("swarm_message:").unwrap_or("").trim();
    // ...
    let final_text = super::run_swarm_message(agent.clone(), msg).await?;
    return Ok(final_text);
}
```

```
// jcode: crates/jcode-app-core/src/server/debug_jobs.rs:77-95
if trimmed.starts_with("swarm_message_async:") {
    // ...
    let result = super::run_swarm_message(agent.clone(), &msg).await;
```

Meanwhile the swarm tool's own "message" action — the thing an agent actually calls — does something else entirely: it routes a direct message, a channel post, or a broadcast. It never calls `run_swarm_message`. So:

`run_swarm_task/run_swarm_message` is debug/dev tooling — useful for testing the orchestration engine from a socket, but not production behavior a spawned agent can trigger. We won't build this chapter's mental model on it.

The real, agent-facing path — the one this whole chapter is about — is:

```
swarm tool, action: "spawn"  
  → Request::CommSpawn  
  → handle_comm_spawn (server-side)  
  → spawn_swarm_agent
```

`spawn_swarm_agent` lives at `crates/jcode-app-core/src/server/comm_session.rs:557-827`, and it's the one function to actually understand in this chapter.

Step 1: the tool call itself

`swarm` is a single, multi-action tool — one schema, many action values. Its name is asserted directly in the `Tool` trait implementation:

```
// jcode: crates/jcode-app-core/src/tool/communicate.rs:1939-1946  
impl Tool for CommunicateTool {  
    fn name(&self) -> &str {  
        "swarm"  
    }  
}
```

The full action list runs to nearly forty values (`crates/jcode-app-core/src/tool/communicate.rs:1954-1963`) — plans, channels, the task graph, and more, each belonging to a later chapter. "spawn" is one value among them. The fields that matter for spawning:

```
// jcode: crates/jcode-app-core/src/tool/communicate.rs:2009-2025, 2043-2047
"label": {
  "type": "string", "minLength": 1,
  "description": "Required for spawn. Short label shown on the agent's chip (e.g. 'api reviewer')."
},
"working_dir": { "type": "string", "description": "Optional working directory for spawn." },
"prompt": {
  "type": "string",
  "description": "Initial task/instructions for spawn. Spawning without it creates an idle agent."
},
"initial_message": { "type": "string", "description": "Alias of prompt for spawn; wins when both are set." },
// ...
"spawn_mode": {
  "type": "string",
  "enum": ["visible", "headless", "inline", "auto"],
  "description": "Spawn UI mode: visible terminal, headless, inline gallery, or auto. Defaults to inline."
},
},
```

label is required only when action: "spawn" — enforced with a JSON Schema anyOf instead of a flat top-level required list, specifically so the requirement doesn't leak onto every other action:

```
// jcode: crates/jcode-app-core/src/tool/communicate.rs:2154-2187
// `swarm` is a multi-action tool, so putting `label` in the top-level
// `required` array would incorrectly require it for read/list/message
// and
// every other action. Use mutually exclusive action branches instead...
schema["anyOf"] = json!([
  {
    "type": "object",
    "required": ["action", "label"],
    "properties": {
      "action": { "type": "string", "enum": ["spawn"] },
      // Gemini validates that every `required` name is defined in
      // the same object's `properties` and rejects the whole
      // request otherwise (issue #655), so declare `label` here
      // instead of relying on the parent schema's declaration.
      "label": { "type": "string", "minLength": 1 }
    }
  },
  { "type": "object", "required": ["action"], "properties": { "action":
    { "type": "string", "enum": non_spawn_actions } } }
]);
```

That inline comment is worth pausing on: a real, dated cross-provider bug — Gemini's schema validator rejects a required field it can't find declared locally — shaped this exact bit of JSON Schema. Multi-model tool schemas have their own cross-browser-style portability quirks.

The spawn action handler itself doesn't touch any swarm state directly:

```
// jcode: crates/jcode-app-core/src/tool/communicate.rs:2720-2751
"spawn" => {
    let label = params.required_spawn_label()?;
    let request = Request::CommSpawn {
        id: REQUEST_ID,
        session_id: ctx.session_id.clone(),
        working_dir: params.working_dir.clone(),
        initial_message: params.spawn_initial_message(),
        request_nonce: None,
        spawn_mode: params.spawn_mode.clone(),
        model: params.model.clone(),
        effort: params.effort.clone(),
        label: Some(label),
    };
    match send_request(request).await {
        Ok(ServerEvent::CommSpawnResponse { new_session_id, .. }) if
            !new_session_id.is_empty() => {
            Ok(ToolOutput::new(format!("Spawned new agent: {}",
                new_session_id)))
        }
        // ...
    }
}
```

It builds a typed Request value and awaits a response over an internal channel. The code that parses the LLM's tool-call JSON never reaches into a shared registry directly — it hands off a message and waits.

```

// idiomatic TS equivalent – this code does not exist in jcode
type SpawnRequest = {
  kind: "spawn";
  sessionId: string;
  workingDir?: string;
  initialMessage?: string;
  spawnMode?: "visible" | "headless" | "inline" | "auto";
  label: string;
};

// The tool handler never touches swarm state directly. It posts a
// typed action and awaits a typed reply – like dispatching to a
// Redux-style reducer, or postMessage()-ing a dedicated worker that
// owns the real state.
async function handleSpawnAction(params: { label?: string; workingDir?:
  string; prompt?: string }) {
  if (!params.label) throw new Error("label is required for spawn");
  const request: SpawnRequest = {
    kind: "spawn",
    sessionId: currentSessionId(),
    workingDir: params.workingDir,
    initialMessage: params.prompt,
    label: params.label,
  };
  const response = await sendRequest(request);
  if (response.kind === "spawnResponse" && response.newSessionId) {
    return `Spawned new agent: ${response.newSessionId}`;
  }
  throw new Error("spawn failed");
}

```

Step 2: spawn_swarm_agent, the real work

This is the function the tool's request ultimately reaches — crates/jcode-app-core/src/server/comm_session.rs:557–827, 271 lines. Its signature:

```
// jcode: crates/jcode-app-core/src/server/comm_session.rs:557-579
pub(super) async fn spawn_swarm_agent(
    req_session_id: &str,
    swarm_id: &str,
    working_dir: Option<String>,
    initial_message: Option<String>,
    spawn_mode: Option<SwarmSpawnMode>,
    requested_model: Option<String>,
    requested_effort: Option<String>,
    label: Option<String>,
    sessions: &SessionAgents,
    // ... 12 more shared-state/channel params
) -> anyhow::Result<String> {
```

Walking through it, in order:

1. **Resolve the working directory and auth identity to inherit** — a spawned child inherits the parent's exact provider/model/auth route rather than falling back to config defaults.
2. **Resolve the spawn's UI mode**, falling back to the configured default when the caller didn't specify one (let resolved_spawn_mode = spawn_mode.unwrap_or(agents_config.swarm_spawn_mode);, comm_session.rs:591).
3. **Prepare the startup message** — if the caller gave an initial_message, it's appended with instructions telling the new worker to call swarm report before finishing, so the parent has something to read back later.
4. **Try a visible spawn; fall back to headless** — the mode gate covered in detail below.
5. **Register the new member with the swarm's plan**, but only if the plan already has items or participants, and **broadcast the plan update**.
6. **Register the SwarmMember bookkeeping record** — only on the non-headless-fallback branch (more on this below), then **set the task label**, then **persist state**, before anything asynchronous fires.
7. **Conditionally kick off the first turn as a fire-and-forget task** — covered in its own section below.


```

// idiomatic TS equivalent – this code does not exist in jcode
async function spawnWorker(parentId: string, opts: SpawnOptions):
    Promise<string> {
    const workingDir = resolveSpawnWorkingDir(opts.workingDir);
    const identity = resolveInheritedIdentity(parentId);
    const spawnMode = opts.spawnMode ?? config.defaultSpawnMode;

    const startupMessage = opts.initialMessage
      ? appendCompletionReportInstructions(opts.initialMessage)
      : undefined;

    const { newSessionId, isHeadlessFallback } = await
      attemptSpawn(spawnMode, {
        workingDir, identity, startupMessage,
      });

    if (planHasItemsOrParticipants(swarmId)) {
      registerPlanParticipant(swarmId, newSessionId);
      broadcastPlanUpdate(swarmId);
    }

    if (!isHeadlessFallback) {
      registerMember({ sessionId: newSessionId, swarmId, reportBackTo:
        parentId });
    }
    setMemberTaskLabel(newSessionId, opts.label ?? opts.initialMessage);
    persist();

    if (isHeadlessFallback && startupMessage) {
      void runFirstTurn(newSessionId, startupMessage); // fire-and-forget,
      see below
    }
    return newSessionId;
  }
}

```

The visible-vs-headless branch

```

// jcode: crates/jcode-app-core/src/server/comm_session.rs:622-652
let visible_spawn = match resolved_spawn_mode {
  // Inline workers run in-process like headless ones; the difference
  is
  // purely how the coordinator renders them (a live inline gallery).
  SwarmSpawnMode::Headless | SwarmSpawnMode::Inline => {
    Err( anyhow::anyhow!("headless spawn requested") )
  }
  SwarmSpawnMode::Visible | SwarmSpawnMode::Auto =>
    prepare_visible_spawn_session(...),
};

```

The `Err(...)` here for `Headless/Inline` isn't a real failure — it's a control-flow shortcut into the same fallback branch that a genuine visible-spawn failure would also take:

```
// jcode: crates/jcode-app-core/src/server/comm_session.rs:654-695
let (new_session_id, is_headless_fallback) = match visible_spawn {
    Ok((new_session_id, true)) => Ok((new_session_id, false)),
    Ok( (_, false)) | Err(_) => {
        // ...
        create_headless_session(..., Some(req_session_id.to_string()),
            ...).await
    }
}
}??;
```

Step 3: two different things people call "mode"

Here's a genuine gotcha in the source: jcode has **two separate concepts** both casually called "mode," and mixing them up will make the rest of this chapter confusing.

Mode axis #1 — SwarmSpawnMode: how the worker's UI is rendered

```
// jcode: crates/jcode-config-types/src/lib.rs:643-657
#[derive(Debug, Clone, Copy, Serialize, Deserialize, PartialEq, Eq,
    Default)]
#[serde(rename_all = "lowercase")]
pub enum SwarmSpawnMode {
    /// Open a visible/headed terminal window. This was the historical
    /// default.
    Visible,
    /// Create the worker in-process without opening a terminal window.
    Headless,
    /// Like headless (no terminal window), but the coordinator renders a
    /// live
    /// inline gallery viewport of each worker's streaming output.
    #[default]
    Inline,
    /// Try visible first and fall back to headless if a window cannot be
    /// opened.
    Auto,
}
```

The important plain-English point: **Headless and Inline both mean "the child runs as an async task inside the same OS process as the parent server."** No new process, no new OS thread — just another concurrently running

job. Only `Visible` (and `Auto`'s attempt at it) opens a genuinely separate terminal/process. And since `Inline` is the `#[default]`, that's the common case in practice.

```
// idiomatic TS equivalent – this code does not exist in jcode
type SwarmSpawnMode = "visible" | "headless" | "inline" | "auto";

function attemptSpawn(mode: SwarmSpawnMode, opts: SpawnOpts) {
  if (mode === "headless" || mode === "inline") {
    // headless/inline ~ another async job in THIS process,
    // tracked in a Map<sessionId, Promise<void>> – not a worker_thread.
    return runInProcess(opts);
  }
  // visible/auto ~ actually spawning a subprocess/window –
  // child_process.spawn(...), a meaningfully different primitive.
  return spawnVisibleWindow(opts).catch(() => runInProcess(opts));
}
```

Mode axis #2 — "ad hoc / light / deep": how much recursive spawning is allowed

This is the axis this chapter's title is actually about, and it's **not** a Rust enum at all — there's no enum `Mode { AdHoc, Light, Deep }` anywhere in the codebase. It's three operating modes distinguished by a session's recorded *reasoning effort string*:

```
// jcode: crates/jcode-base/src/prompt.rs:129-131
pub fn is_deep_swarm_effort(effort: &str) -> bool {
    effort.trim().eq_ignore_ascii_case(SWARM_DEEP_EFFORT)
}
```

`SWARM_DEEP_EFFORT` is the literal string `"swarm-deep"`. Its sibling `"swarm"` (`SWARM_EFFORT`) is the **light** rung — swarm orchestration enabled, but without the recursive DAG machinery deep mode unlocks. **Ad hoc** is simply neither sentinel: ordinary spawning, still allowed by default, since a first generation of spawns isn't gated by effort at all — only *recursive* (grandchild) spawning is.

A session's current effort is tracked in a small side-table, deliberately kept separate from the agent's own lock to avoid a deadlock:

```
// jcode: crates/jcode-app-core/src/session_effort.rs:1-19
///! The swarm task-graph seed handler runs on the server socket thread
///! while the
///! *seeding* agent is blocked inside its `swarm` tool call holding its
///! own agent
///! lock. That means the handler cannot read the seeder's effort via the
///! agent
///! mutex without deadlocking. This tiny side-table is updated whenever
///! an agent's
///! effort changes (cheap string writes) so server handlers can learn a
///! session's
///! effort by id without taking any agent lock.
```

That's a nice, concrete "why a side-index instead of the obvious lookup" example: the obvious path (ask the agent object for its own effort) would deadlock, because the socket handler needs the answer *while* the agent is still holding its own lock inside the very tool call that triggered the question. A cheap, separately-locked side-table sidesteps that entirely.

The actual gate

The mode-gate and the member cap both live inside one function, `ensure_spawn_coordinator_swarm`, `comm_session.rs:1234-1431` — 198 lines, three independent checks.

Check (a): only the root may spawn recursively, and it's the *root's* effort that's checked, not the caller's own.

```
// jcode: crates/jcode-app-core/src/server/comm_session.rs:1330-1348
// Light and ad hoc swarms are deliberately one-level fan-out: only the
// root
// session may create workers. Recursive spawning is an explicit deep-
// swarm
// capability, keyed from the root's effort rather than the requesting
// child's effort so a worker cannot opt itself into unbounded growth.
if !is_root {
    let root_is_deep =
        crate::session_effort::session_effort(&root_session_id)
            .as_deref()
            .is_some_and(crate::prompt::is_deep_swarm_effort);
    if !root_is_deep {
        // reject: "Recursive swarm spawning is disabled for light and ad
        // hoc swarms..."
        return None;
    }
}
```

This is a genuinely good access-control pattern, worth naming plainly: the check authorizes against a capability derived from the **root of the call chain**, not a value the requester itself controls. A worker can't declare "I'm running at deep effort" to unlock its own recursive spawning — only the root's recorded state counts.

```
// idiomatic TS equivalent – this code does not exist in jcode
function canSpawnRecursively(requester: SwarmMember, rootSessionId:
    string): boolean {
    if (requester.isRoot) return true; // roots always get a first
        generation
    // Authorize against the ROOT's capability, never the caller's own
        claim –
    // a worker cannot self-elevate by lying about its own effort.
    return isDeepSwarmEffort(sessionEffort(rootSessionId));
}
```

Check (b): an absolute hard cap, 1000 live members, enforced even if the configurable limit below is turned off:

```
// jcode: crates/jcode-app-core/src/server/comm_session.rs:1350–1361
if live_member_count >= super::MAX_SWARM_MEMBERS {
    // reject: "Swarm member limit reached (hard max
        {MAX_SWARM_MEMBERS})..."
    return None;
}
```

```
// jcode: crates/jcode-app-core/src/server/swarm.rs:26–32
/// Maximum number of live members (agents) in a single swarm. Re-
    exported from
/// `jcode_swarm_core` so the server, tools, and prompts all agree on the
    one
/// runaway-prevention cap for the task-graph model. Normal and light
    swarms are
/// root-only, one-level fan-out. Deep-swarm roots may create recursive
    trees with
/// no depth limit, but both the configurable live-worker budget and this
    absolute
/// cap still apply.
pub(super) use jcode_swarm_core::MAX_SWARM_MEMBERS;
```

Check (c): a second, configurable, lower budget on top of the hard cap — an operator-tunable "don't exhaust the machine's RAM" limit that used to apply only to one code path and got widened after a real bug:

```
// jcode: crates/jcode-app-core/src/server/comm_session.rs:1363-1378
// `swarm_max_concurrent_agents` is the machine-safety budget shared by
// run_plan and deep recursive spawning. Previously only run_plan obeyed
// it,
// so nested agents could grow to the 1000-member hard cap and exhaust
// RAM.
let live_agent_limit = (configured_live_agent_limit > 0)
    .then(|| configured_live_agent_limit.min(super::MAX_SWARM_MEMBERS));
```

Two caps, two purposes: 1000 is an absolute safety valve; the configurable budget is a smaller, operator-set ceiling for machines that can't run a thousand concurrent agents. Not every counted member stays counted forever — finished, stopped, or failed members stop consuming capacity:

```
// jcode: crates/jcode-app-core/src/server/swarm.rs:234-236
pub(super) fn member_consumes_swarm_capacity(member: &SwarmMember) ->
    bool {
    !member_status_is_terminal(&member.status)
}
```

```
// idiomatic TS equivalent — this code does not exist in jcode
const MAX_SWARM_MEMBERS = 1000; // absolute hard ceiling, never
// configurable

function canSpawnAnother(swarm: SwarmState, configuredLimit: number):
    boolean {
    const liveCount = [...swarm.members.values()]
        .filter(m => !isTerminalStatus(m.status)).length;
    if (liveCount >= MAX_SWARM_MEMBERS) return false;
    const softLimit = configuredLimit > 0 ? Math.min(configuredLimit,
        MAX_SWARM_MEMBERS) : Infinity;
    return liveCount < softLimit;
}
```

The fire-and-forget first turn

When a headless/inline spawn includes a startup message, spawn_swarm_agent kicks off the first turn without waiting for it:

```
// jcode: crates/jcode-app-core/src/server/comm_session.rs:772-822
tokio::spawn(async move {
    update_member_status(&sid_clone, "running", ..., ...).await;
    let event_tx = super::session_event_fanout_sender(...);
    let start_message_index = { let agent = agent_arc.lock().await;
        agent.message_count() };
    let result = process_message_streaming_mpsc(Arc::clone(&agent_arc),
        &initial_msg, vec![], None, event_tx).await;
    let completion_report = if result.is_ok() { /* ... */ } else { None
    };
    let (new_status, new_detail) = match result {
        Ok(()) => ("ready", None),
        Err(ref error) => ("failed",
            Some(truncate_detail(&error.to_string(), 120))),
    };
    update_member_status_with_report(&sid_clone, new_status, new_detail,
        completion_report, ...).await;
});
```

`tokio::spawn(async move { ... })` starts a task running concurrently and hands back a `JoinHandle` — which is immediately dropped here, never `.awaited`. That's exactly what "fire-and-forget" means in async Rust: the outer function returns the new session id right away, without waiting to see how the first turn goes. The only way anything downstream learns how it went is by later reading the member's status or completion report — never through a `Result` handed back to whoever triggered the spawn.

The direct TypeScript analogue is one every linter warns about for a reason: calling an `async` function without awaiting it — a **floating promise**. If it rejects and nothing is listening, the error vanishes silently unless a fallback is wired up (jcode's status/report fields, here).

```
// idiomatic TS equivalent – this code does not exist in jcode
function fireFirstTurn(sessionId: string, initialMessage: string) {
    updateMemberStatus(sessionId, "running");
    // No `await` here – deliberately fire-and-forget. Most TS linters
    // (no-floating-promises) would flag this exact line by default,
    // because errors below silently disappear unless handled here.
    runFirstTurn(sessionId, initialMessage)
        .then(() => updateMemberStatusWithReport(sessionId, "ready", null,
            latestReport(sessionId)))
        .catch(err => updateMemberStatusWithReport(sessionId, "failed",
            String(err).slice(0, 120), null));
    // spawnWorker() already returned sessionId before this ever settles.
}
```

This is a deliberate tradeoff, not an oversight: whoever spawned this worker gets an id back immediately and has to poll or subscribe for status, because there's no synchronous "did it work" signal.

Ancestry: where `report_back_to_session_id` actually gets set

Chapter 1 introduced `report_back_to_session_id` as the one field that lets the whole spawn tree be derived instead of stored. Here's where it's actually written.

For a visible spawn, the new `SwarmMember` record is built directly:

```
// jcode: crates/jcode-app-core/src/server/comm_session.rs:502-519
// (relevant field only)
SwarmMember {
    session_id: session_id.to_string(),
    // ...
    report_back_to_session_id:
        report_back_to_session_id.map(str::to_string),
    // ...
    role: "agent".to_string(),
    // ...
}
```

called with `Some(req_session_id)` — the id of whichever session's swarm tool call triggered the spawn.

For the default, headless/inline path, the same value is threaded through as a parameter to `create_headless_session`:

```
// jcode: crates/jcode-app-core/src/server/headless.rs:37-54 (signature only)
pub(super) async fn create_headless_session(
    // ...
    report_back_to_session_id: Option<String>,
    memory_scope: HeadlessMemoryScope,
) -> Result<String> {
```

called with `Some(req_session_id.to_string())` at that exact parameter position. That confirms the parent edge gets threaded into the headless path correctly too — though the exact line inside `headless.rs` where it lands in the constructed `SwarmMember` wasn't traced this pass, so it's worth treating as "proven by the call site, not pinned to a specific line inside that function."


```
// idiomatic TS equivalent – this code does not exist in jcode
function registerMember(sessionId: string, swarmId: string, reportBackTo:
    string | null) {
    members.set(sessionId, {
        sessionId,
        swarmId,
        reportBackToSessionId: reportBackTo, // parentMap.set(newId,
            callerId)
        role: "agent",
        status: "starting",
    });
}
```

Reparenting when a member leaves

If jcode only ever added members, the parent-pointer design from Chapter 1 would be trivial. The interesting part is what happens when a member with children of its own **departs** the swarm — those children can't be left pointing at a session that no longer exists. `remove_session_from_swarm` (swarm.rs:995–1213, the largest function in the file) handles it, and its own doc comment states the stakes plainly:

```
// jcode: crates/jcode-app-core/src/server/swarm.rs:1115–1121
// Reparent the departing member's direct children so the spawn tree
//   never
// holds dangling report-back edges. Orphaned subtrees would otherwise
// silently change ownership semantics: stop permissions, subtree
//   broadcast
// scope, and completion report-back all walk this chain. Children are
// attached to their grandparent when it is still a live member of this
// swarm, otherwise to the current coordinator, otherwise they become
// roots (report_back_to_session_id = None).
```

The fallback chain, computed directly:

```
// jcode: crates/jcode-app-core/src/server/swarm.rs:1122-1142
let fallback_parent: Option<String> = {
    let grandparent_is_live = if let Some(ref parent) = departing_parent
    {
        parent != session_id && {
            let members = swarm_members.read().await;
            members.get(parent).is_some_and(|member|
                member.swarm_id.as_deref() == Some(swarm_id))
        }
    } else {
        false
    };
    if grandparent_is_live {
        departing_parent.clone()
    } else {
        let coordinators = swarm_coordinators.read().await;
        coordinators.get(swarm_id).filter(|coordinator|
            coordinator.as_str() != session_id).cloned()
    }
};
```

And the loop that actually rewrites every affected child:

```
// jcode: crates/jcode-app-core/src/server/swarm.rs:1143-1156
let mut reparented: Vec<String> = Vec::new();
{
    let mut members = swarm_members.write().await;
    for member in members.values_mut() {
        if member.swarm_id.as_deref() == Some(swarm_id)
            && member.report_back_to_session_id.as_deref() ==
            Some(session_id)
        {
            member.report_back_to_session_id = fallback_parent
                .clone()
                .filter(|parent| parent != &member.session_id);
            reparented.push(member.session_id.clone());
        }
    }
}
```

Precisely: **grandparent, if it's still a live member of the same swarm** → **else the current coordinator (excluding the departing session itself)** → **else None, promoted to root**. The trailing `.filter(...)` guards against a self-loop, where a member could otherwise end up listed as its own parent.

One more piece the design docs don't mention: if the *departing* member was itself the coordinator, a new one is elected — the lowest session id among the swarm's remaining live, non-headless members — before reparenting even

happens:

```
// jcode: crates/jcode-app-core/src/server/swarm.rs:1056-1070
let new_coordinator = {
    let swarms = swarms_by_id.read().await;
    let members = swarm_members.read().await;
    swarms.get(swarm_id).and_then(|swarm| {
        swarm.iter().filter_map(|id| {
            members.get(id).filter(|member| !member.is_headless).map(|_|
                id.clone())
        }).min()
    })
};
```

So departure teardown has to fix up two independent kinds of "who's in charge" in one pass: the coordinator slot for the whole swarm, and the parent pointer for each of the departing member's own children.

```

// idiomatic TS equivalent – this code does not exist in jcode
function removeMember(sessionId: string, swarm: SwarmState) {
  // 1. If the departing member was coordinator, elect a new one first –
  //     lowest session id among live, non-headless members.
  if (swarm.coordinators.get(swarm.id) === sessionId) {
    const next = [...swarm.members.values()]
      .filter(m => !m.isHeadless && m.sessionId !== sessionId)
      .map(m => m.sessionId).sort()[0];
    if (next) {
      swarm.coordinators.set(swarm.id, next);
      swarm.members.get(next)!.role = "coordinator";
    }
  }

  // 2. Reparent every direct child of the departing member: grandparent
  //     if
  //     still live, else the coordinator, else promote to root.
  const parent = swarm.members.get(sessionId)?.reportBackToSessionId ??
    null;
  const grandparentLive = parent !== null && parent !== sessionId
    && swarm.members.get(parent)?.swarmId === swarm.id;
  const fallback = grandparentLive ? parent
    : (swarm.coordinators.get(swarm.id) !== sessionId ?
      swarm.coordinators.get(swarm.id) ?? null : null);

  for (const member of swarm.members.values()) {
    if (member.swarmId === swarm.id && member.reportBackToSessionId ===
      sessionId) {
      member.reportBackToSessionId = fallback === member.sessionId ? null
        : fallback;
    }
  }
  swarm.members.delete(sessionId);
}

```

Structurally, this whole section is "removing a node from a tree keyed by parent-pointers on the children, not child-lists on the parent" — removal means scanning for every node whose parent pointer equals the removed id and rewriting it. Coordinator election is a plain min/sort over live candidate ids. Neither needs anything Rust-specific to understand.

What this chapter didn't cover

Two things worth flagging rather than glossing over: the exact line inside `headless.rs` where the `SwarmMember` record gets constructed for headless/inline spawns wasn't traced this pass — only that the parent id is correctly passed into `create_headless_session`. And the relationship between the effort-string mode

gate covered here and a separate `Deep/Light` mode enum used by the task-graph machinery (Chapter 6 territory) hasn't been confirmed — they may or may not be cross-checked against each other. Treat them as two different concepts until proven otherwise.

Next: Chapter 3 covers what happens *after* a spawn — the three distinct fan-out/fan-in patterns jcode uses once multiple agents are actually running concurrently.

Chapter 3 — Fan-out / Fan-in Patterns

Every "run several things at once, then do something with the results" problem looks the same from a distance. Up close, jcode actually contains three genuinely different answers to it, and picking the wrong one produces code that's either slower than it needs to be or subtly wrong under load.

This chapter walks through all three, back to back, using the exact code that ships in the repo. The goal isn't "here are three ways to do concurrency" as a grab-bag — it's to give you a decision procedure. By the end you should be able to look at a new problem and know which of the three shapes it wants.

The three questions that separate them:

1. **Is the set of parallel work fixed before anything starts, or discovered as things run?**
2. **Do you need the whole batch's result at once, or do you want to react as each piece finishes?**
3. **Is "fan-in" a single call that blocks until everything is done, or a long-lived loop that wakes up when something happens?**

Keep those three questions in your head — we'll come back to them in the decision table at the end.

Pattern 1: `try_join_all` planner fan-out

The plain-English problem: you want an LLM to break a task into a handful of independent pieces, run all the pieces at the same time, then feed all the results back into one final "combine everything" step. This is the mental picture most people have when they hear "swarm of agents": a planner splits the work, N workers run concurrently, one final step stitches the answers together.

One important caveat before we look at the code: this exact function, `run_swarm_message`, is reachable only from two debug-socket commands (`swarm_message:` and `swarm_message_async:`) — it is **not** what happens when a production agent calls the `swarm` tool. The live spawn mechanism (covered in

Chapter 2) is a different code path entirely. We're still using this function as the chapter's flagship example, because it's real, shipped code that demonstrates the pattern cleanly — just don't walk away thinking this is how a normal agent turns out work.

With that said, here's the shape. The function does three sequential LLM calls, with exactly one concurrent step sandwiched in the middle:

1. Ask the LLM to plan: "break this into 2-4 subtasks."
2. Run all the subtasks **at once**.
3. Ask the LLM to integrate all the subtask outputs into one final answer.

Steps 1 and 3 both run on the same locked coordinator agent — they're not concurrent with anything. Step 2 is the interesting part:

```
// jcode: crates/jcode-app-core/src/server/swarm.rs:1657-1671
let task_futures = tasks.iter().map(|task| {
    let agent = agent.clone();
    let working_dir_hint = working_dir_hint.clone();
    let description = task.description.clone();
    let prompt = format!("{working_dir_hint}{}", task.prompt);
    let subagent_type = task
        .subagent_type
        .clone()
        .unwrap_or_else(|| "general".to_string());
    async move {
        let output = run_swarm_task(agent, &description, &subagent_type,
            &prompt).await?;
        Ok:::<(String, String), anyhow::Error>((description, output))
    }
});
let task_outputs = try_join_all(task_futures).await?;
```

Two things happen here. First, `tasks.iter().map(...)` builds an iterator of **futures** — nothing has actually started running yet, this is just constructing the work items (async blocks in Rust are lazy; they don't run until polled). Second, `try_join_all` is what actually launches all of them concurrently and waits.

Each of those futures, when it eventually runs, calls `run_swarm_task`, which forks a brand-new session and agent, inherits the parent's exact model/provider/auth identity so the worker doesn't silently fall back to a config default, and strips `subagent/task/todo*` from the child's tool set — a forked worker in this pattern cannot itself spawn more workers. It runs exactly one prompt to completion and returns a `Result<String>`.

Why `try_join_all` specifically: it needs every future to resolve to the same `Result<T, E>` shape, and it fails fast — the moment any one subtask returns an error, the whole call returns that error immediately, and the other still-in-flight futures are simply dropped (which, on tokio, stops polling them; nothing explicitly cancels or cleans them up). That's the correct behavior here specifically because step 3, the integration call, has no sensible way to combine "2 of the 3 planned subtasks succeeded" into a coherent answer. The plan assumed 3 outputs; if you only have 2, there's nothing good to do with them, so failing the whole operation immediately is the right call.

The TypeScript equivalent

```
// idiomatic TS equivalent – this code does not exist in jcode
interface SwarmTaskSpec {
  description: string;
  prompt: string;
  subagentType?: string;
}

async function runSwarmMessage(
  coordinator: Agent,
  message: string,
): Promise<string> {
  // Step 1: plan (sequential, runs on the coordinator itself)
  const planText = await
    coordinator.runOnceCapture(buildPlannerPrompt(message));
  const tasks: SwarmTaskSpec[] = parseSwarmTasks(planText) ?? [
    { description: "Main task", prompt: message },
  ];

  // Step 2: fan-out + fan-in – the concurrent step
  const taskOutputs = await Promise.all(
    tasks.map(async (task) => {
      const worker = await forkWorkerAgent(coordinator, task.subagentType
        ?? "general");
      const output = await worker.runOnceCapture(task.prompt);
      return [task.description, output] as const;
    }),
  );

  // Step 3: integrate (sequential, back on the coordinator)
  return coordinator.runOnceCapture(buildIntegrationPrompt(taskOutputs));
}
```

`Promise.all` is almost a literal translation of `try_join_all`: same fail-fast semantics, same "reject as soon as one rejects." The one nuance worth internalizing: `Promise.all` doesn't actually *cancel* the sibling promises when one

rejects — they keep running in the background, their results are just ignored. That's exactly the same "fire, don't truly cancel" behavior as Rust's dropped future. If you assumed `Promise.all` (or `try_join_all`) stops the other work, both languages will surprise you the same way.

Pattern 2: FuturesUnordered intra-agent batch tool

The plain-English problem: a single agent turn wants to fire off several independent tool calls at once — say, reading five files — as one "batch" tool invocation, and it wants to show live "3 of 5 done" progress to the UI as each one finishes, not just block until everything is done.

This is a different shape from Pattern 1 in a way that matters: these aren't LLM-planned subagents with their own sessions, they're plain tool calls going through the same tool registry. And the defining requirement — observing completion *incrementally, in whatever order things actually finish* — is exactly what `try_join_all`/`Promise.all` cannot give you. Both of those only resolve once, with everything, after all of it is done.

```
// jcode: crates/jcode-app-core/src/tool/batch.rs:282-295
let mut stream: futures::stream::FuturesUnordered<_> = subcalls
    .iter()
    .map(|(i, tool_name, parameters)| {
        let registry = self.registry.clone();
        let i = *i;
        let tool_name = tool_name.clone();
        let parameters = parameters.clone();
        let sub_ctx = ctx.for_subcall(format!("batch-{}-{}", i + 1,
            tool_name.clone()));
        async move {
            let result = registry.execute(&tool_name, parameters,
                sub_ctx).await;
            (i, tool_name, result)
        }
    })
    .collect();
```

Each future captures its own original index `i` and hands it back alongside the result — that's the trick that lets the code recover original order later, even though the stream itself will yield things in *completion* order, not submission order.

Then the drain loop:

```
// jcode: crates/jcode-app-core/src/tool/batch.rs:300-317
while let Some((i, tool_name, result)) = stream.next().await {
    // ... publishes one BusEvent::BatchProgress per completion
}
```

Each iteration through this loop corresponds to exactly one sub-call finishing, in whatever order they actually complete in. Each iteration publishes a `BusEvent::BatchProgress` update — incremented count, name of the thing that just finished — onto the global event bus (the same bus covered in Chapter 7; progress from this purely intra-turn concurrency mechanism is surfaced through the same process-wide channel everything else in the UI listens to).

Once the loop drains, the code does one more thing worth remembering:

```
results.sort_by_key(|(i, _, _)| *i) // jcode: tool/batch.rs:319
```

It re-sorts back into original submitted order before formatting the final text handed back to the LLM. Getting completion-order efficiency for progress reporting *and* deterministic output order for the final result isn't a contradiction — it just costs one sort at the end. There's also a hard cap: `MAX_PARALLEL = 10` (line 10 of the same file) rejects batches that are too big, and every self-nested batch call inside a batch call is rejected outright, so this can't be used to build unbounded fan-out.

The TypeScript equivalent

There's no single built-in with exactly this shape — `Promise.allSettled` gets you completion tolerance but still only resolves once, at the end. The honest idiomatic equivalent is a small hand-rolled drain loop:

```

// idiomatic TS equivalent – this code does not exist in jcode
async function* runBatchIncremental<T>(  
  jobs: Array<() => Promise<T>>,  
) : AsyncGenerator<{ index: number; result: T }> {  
  const pending = new Map<number, Promise<{ index: number; result: T }>>  
    ();  
  
  jobs.forEach((job, index) => {  
    pending.set(  
      index,  
      job().then((result) => ({ index, result })),  
    );  
  });  
  
  while (pending.size > 0) {  
    const winner = await Promise.race(pending.values());  
    pending.delete(winner.index);  
    yield winner;  
  }  
}  
  
// usage: publish progress as each tool call finishes  
const results: unknown[] = [];  
let completed = 0;  
for await (const { index, result } of runBatchIncremental(toolCalls)) {  
  completed++;  
  bus.publish({ type: "BatchProgress", completed, total: toolCalls.length  
    });  
  results[index] = result; // restore original order as we go  
}

```

Worth flagging honestly: naive re-racing with `Promise.race` inside a loop like this is $O(n)$ work per completion, because each call to `race` re-wraps every remaining promise. It's close enough in spirit to `FuturesUnordered` to teach the idea, but a production implementation would want a counter/resolver pattern instead of literally calling `Promise.race` in a loop. The generator interface is the more important part to internalize: for `await...of` giving you one result at a time, in completion order, is the actual shape `FuturesUnordered` provides.

Pattern 3: event-driven fan-in via broadcast + `tokio::select!`

The plain-English problem: you want to wait until some (or all) of a set of *already-running* agents reach a target status — but give up after a deadline, and don't burn CPU polling in a tight loop while you wait.

This is different from both patterns above in a structural way: the "workers" here weren't started by this function at all. They're independent swarm members that were spawned earlier and are doing their own work. This function only *waits* on them — there's no fixed, bounded set of futures to collect like Pattern 2, and nothing to fan out like Pattern 1.

```
// jcode: crates/jcode-app-core/src/server/comm_await.rs:271-300
tokio::select! {
    _ = tokio::time::sleep_until(deadline) => {
        let summary = timeout_summary(&member_statuses);
        finalize_await(&await_members_runtime, &state, false,
            member_statuses, summary).await;
        return;
    }
    event = event_rx.recv() => {
        match event {
            Ok(event) => {
                if event.swarm_id.as_deref() != Some(swarm_id.as_str()) {
                    continue;
                }
            }
            Err(broadcast::error::RecvError::Lagged(n)) => {
                // Dropped events are recoverable: the loop re-reads
                // member statuses from shared state at the top, so
                // just keep watching instead of orphaning the wait.
                continue;
            }
            Err(broadcast::error::RecvError::Closed) => {
                await_members_runtime.clear_active(&key).await;
                return;
            }
        }
    }
}
```

This lives inside a loop (the full function is `spawn_or_resume_await_members`, `comm_await.rs:209-303`), run as a detached background task via `tokio::spawn`. Each time around the loop, two things race: a timeout, and the next incoming `SwarmEvent` off a `broadcast::Receiver`.

Here's the design point that matters most: **the event itself is never trusted as the source of truth**. Every iteration starts by re-reading every watched member's current status directly from shared state (`awaited_member_statuses`) — not from the event payload — and checking whether the wait's condition ("any" or "all" members reached the target status) is now satisfied. The event's only job is to wake the loop up sooner than the next timeout tick would.

That design choice is what makes the `Lagged(n)` branch safe to just log and continue past, rather than treat as an error. A `broadcast::Receiver`'s `recv()` returns `Err(Lagged(n))` when the channel's ring buffer overwrote `n` messages this particular receiver hadn't read yet — a slow subscriber missing events under backpressure. Because the loop always re-derives "are we done" from live shared state rather than from the stream of events, a missed event costs at most a slightly stale wakeup, never an incorrect final answer. A comment right at the `Lagged` arm says it plainly: dropped events are recoverable because the loop re-reads member statuses from shared state at the top, so it just keeps watching instead of orphaning the wait.

Contrast the failure model with Pattern 1: `try_join_all` produces a `Result` and short-circuits on error. This function isn't producing a `Result` at all — it's a status-matching wait with a deadline. The `"any"/"all"` mode choice is semantically close to `Promise.race` vs. `Promise.all`, but it's implemented as a polling/event-driven loop against state owned somewhere else, not a combinator over futures you started yourself. You reach for this pattern specifically when the things you're waiting on are not futures you hold — they're independent, already-running work whose progress lives in shared mutable state.

One more connection worth knowing about, since it bridges into Chapter 7: when a wait finishes and was started as a background wait with notification requested, `finalize_await` additionally publishes a `BusEvent::SwarmAwaitCompleted` onto the *global* event bus — a different, process-wide channel from the per-swarm `broadcast::Sender<SwarmEvent>` this function subscribes to. That's the exact hand-off point between the swarm-scoped fan-in mechanism in this chapter and the global bus covered next.

The TypeScript equivalent

```
// idiomatic TS equivalent – this code does not exist in jcode
async function awaitMembers(
  swarmEvents: EventEmitter,
  swarmId: string,
  deadlineMs: number,
  isSatisfied: () => boolean, // re-derives from live shared state
): Promise<{ satisfied: boolean }> {
  return new Promise((resolve) => {
    const timer = setTimeout(() => {
      cleanup();
      resolve({ satisfied: false });
    }, deadlineMs);

    const onEvent = (event: { swarmId: string }) => {
      if (event.swarmId !== swarmId) return; // filter, like the swarm_id
      // check
      // Never trust the event payload directly – always re-check live
      // state.
      if (isSatisfied()) {
        cleanup();
        resolve({ satisfied: true });
      }
      // otherwise: fall through and keep waiting, same as `continue`
    };

    function cleanup() {
      clearTimeout(timer);
      swarmEvents.off("swarm-event", onEvent);
    }

    swarmEvents.on("swarm-event", onEvent);

    // Check once up front in case we're already satisfied.
    if (isSatisfied()) {
      cleanup();
      resolve({ satisfied: true });
    }
  });
}
```

The closest built-in analogy for `broadcast::Receiver` is an `EventEmitter` sitting on a bounded ring buffer that can drop old events under backpressure — except Rust's channel tells the receiver explicitly via `Lagged(n)`, where Node's `EventEmitter` would just silently and invisibly drop them. The one property worth naming explicitly, because it's the load-bearing correctness rule of the whole pattern: **always re-derive "are we done" from source-of-truth**

state when you wake up — never trust the event payload as authoritative. That's what makes a lossy channel safe to build a correctness-sensitive wait on top of.

When would you reach for each one

	Pattern 1: <code>try_join_all</code>	Pattern 2: <code>FuturesUnordered</code>	Pattern 3: broadcast + select!
What you're waiting on	Futures you just started, whose <i>set</i> is fixed up front (a parsed LLM plan)	Futures you just started, whose count is small/bounded (≤ 10)	Already-running, independently-owned agents you didn't start
When you find out about progress	Only once, all at once, at the end	Incrementally, one at a time, in completion order	Incrementally via wake-ups, re-verified against shared state each time
Failure behavior	Short-circuits on first error	Every result collected individually — no short-circuit	No <code>Result</code> at all — a status-matching wait with a timeout
Needs a deadline?	No	No	Yes — the only one of the three built around a timeout
Source citation	<code>swarm.rs</code> :1671 (the call), 1657–1670 (fan-out build)	<code>tool/batch.rs</code> :282–295 (build), 300–317 (drain)	<code>comm_await.rs</code> :271–300 (<code>select!</code>), 209–303 (whole function)
TS analogy	<code>Promise.all</code>	Hand-rolled incremental drain (race-in-a-loop or async generator)	<code>EventEmitter</code> + timeout race, re-deriving state on every wake

If the set of parallel work is known up front and you need all of it before proceeding: **Pattern 1**. If you're firing off a small bounded batch and want to react as pieces land: **Pattern 2**. If you're waiting on work that's already running somewhere else, with a deadline, and shouldn't poll: **Pattern 3**.

A quick note on what this chapter didn't cover: the notes this chapter draws from flagged one honest gap — nobody traced whether an already-spawned child session's on-disk state gets cleaned up when `try_join_all` short-circuits and drops its sibling futures. What we know for certain is the language-level fact (a

dropped future stops being polled); whether jcode does anything further to clean up an abandoned child session is not something this chapter claims one way or the other.

Chapter 4 — Structured Concurrency & Cancellation

Every agent swarm eventually asks the same two questions: *when I spawn background work, how do I know it actually stops when I tell it to?* and *when a user hits cancel, how fast and how reliably does that reach the work in flight?*

jcode answers these with two separate, hand-built primitives that live at different layers of the system:

- `RuntimeTaskScope` — makes sure the server never loses track of a background task it spawned, and can shut all of them down cleanly.
- `InterruptSignal` — lets a user's Esc/Ctrl+C reach a running agent turn fast, without the agent loop wasting cycles constantly polling "am I cancelled yet?"

Neither is exotic. Both are built from a small number of off-the-shelf concurrency primitives, combined in a way that closes specific, nameable bugs. That combination — and *why* it's needed at all — is what this chapter teaches.

Part A: `RuntimeTaskScope` — never losing track of a spawned task

The problem

Picture a server that accepts many client connections and spawns a background task to handle each one. The obvious way to spawn a task is also the easiest way to create a bug: you fire off the task, you don't need its result right this second, so you throw away the handle you'd use to check on it. Now nobody owns that task. Nobody can ask "are you still running?" and nobody can tell it to stop when the server shuts down. The task keeps going — against a server that, as far as anyone else is concerned, is already gone.

Structured concurrency is the fix as a discipline: a parent that spawns child tasks must be able to enumerate every child it has spawned, and must wait for (or forcibly stop) every one of them before the parent itself is considered finished.

RuntimeTaskScope is jcode's implementation of that discipline for the server's connection-handling tasks.

It's built from two ingredients:

- a **join set** — a live collection of "tasks I've spawned that I can wait on, together or one at a time" (Rust's `tokio::task::JoinSet`)
- a **cancellation token** — a cheap, shareable "please stop" flag that can propagate from a parent to every child it hands a copy to (Rust's `tokio_util::sync::CancellationToken`)

Here's the struct, with its own doc comment left in because it states the danger plainly:

```
// jcode: crates/jcode-app-core/src/server/runtime.rs:27-37
/// Owns every connection task spawned by a server runtime.
///
/// Dropping a `JoinHandle` detaches its task, so accepting a connection
/// must not
/// discard the handle. This scope gives the accept loops and their
/// children one
/// cancellation boundary and lets server shutdown wait until all
/// children have
/// observed cancellation and released their resources.
#[derive(Default)]
struct RuntimeTaskScope {
    cancellation: CancellationToken,
    tasks: Mutex<JoinSet<()>>,
}
```

A `JoinSet<()>` behaves a bit like `Promise.all`, except you can keep adding tasks to it after others are already running, and you can drain it incrementally ("give me whichever one finished next") instead of only getting one combined result at the very end. `CancellationToken` is cheap to clone, and calling `.child_token()` on one produces a *linked* token that fires automatically whenever its parent fires — that's how one shutdown call reaches every spawned task without the scope having to loop over them and notify each one by hand.

TS teaching equivalent: `AbortController`/`AbortSignal` cover the "please stop" half, but there's a real gap worth naming: `AbortSignal` has no built-in `child_token()`. Chaining parent-to-child abort behavior in JS means manually listening for the parent's abort event and calling `.abort()` on a child controller yourself. For the "join set" half, there's no single built-in — the idiomatic shape is a Set of in-flight promises you add to on spawn and remove from as they settle.

Registering a task under the scope

```
// jcode: crates/jcode-app-core/src/server/runtime.rs:40-59
async fn spawn<F, Fut>(&self, task: F) -> bool
where
    F: FnOnce(CancellationToken) -> Fut,
    Fut: Future<Output = ()> + Send + 'static,
{
    if self.cancellation.is_cancelled() {
        return false;
    }
    let mut tasks = self.tasks.lock().await;
    while let Some(result) = tasks.try_join_next() {
        log_task_completion(result);
    }
    if self.cancellation.is_cancelled() {
        return false;
    }
    tasks.spawn(task(self.cancellation.child_token()));
    true
}
```

Before accepting a new task it: refuses outright if the scope is already shutting down; opportunistically drains any already-finished tasks out of the set (housekeeping done on the way in, rather than a separate cleanup pass); checks cancellation *again*, in case shutdown started while it was draining; then spawns, handing the new task its own linked child token so it can watch for cancellation itself. It returns a `bool` — whether the task was actually accepted — so a caller can react to "sorry, we're shutting down" instead of silently losing the work.

(One Rust-specific detail, for context and then set aside: the bound `Fut: Future<Output = ()> + Send + 'static` is the compiler's guarantee that the task is safe to run on any of the runtime's worker OS threads and doesn't secretly depend on local stack data that might disappear first. This isn't a *problem* that exists in single-threaded JS at all — there's nothing to guarantee.)

```
// idiomatic TS equivalent – this code does not exist in jcode
class RuntimeTaskScope {
  private controller = new AbortController();
  private tasks = new Set<Promise<void>>();

  async spawn(task: (signal: AbortSignal) => Promise<void>):
    Promise<boolean> {
    if (this.controller.signal.aborted) return false;

    const promise = task(this.controller.signal).finally(() => {
      this.tasks.delete(promise);
    });
    this.tasks.add(promise);
    return true;
  }

  async shutdown(): Promise<void> {
    this.controller.abort();
    const inFlight = [...this.tasks];
    await Promise.allSettled(inFlight);
  }
}
```

The deadlock this code was written to avoid

This is the best single "here's a real, subtle bug and the fix for it" moment in the file:

```
// jcode: crates/jcode-app-core/src/server/runtime.rs:61-74
async fn shutdown(&self) {
  self.cancellation.cancel();
  // Drain the set before awaiting children. An accept task may already
  // be
  // waiting to register a just-accepted connection; leaving the mutex
  // held while joining would deadlock that task. Once cancelled, any
  // late registration observes cancellation and is rejected.
  let mut tasks = {
    let mut owned_tasks = self.tasks.lock().await;
    std::mem::take(&mut *owned_tasks)
  };
  while let Some(result) = tasks.join_next().await {
    log_task_completion(result);
  }
}
```

If shutdown held the lock on the task set *while also* waiting for each child task to finish, and one of those children was itself blocked trying to acquire that same lock (to register itself as a new task), the two would wait on each other forever —

a lock-held-across-await deadlock. The fix, `std::mem::take(&mut *owned_tasks)`, swaps the whole `JoinSet` out for an empty one while briefly holding the lock, then releases the lock immediately and works with its own private copy from then on. It's the concurrency equivalent of "don't hold the front door open while everyone still inside leaves — take a copy of the guest list, close the door, then check people off your copy."

Worth saying plainly: this exact hazard doesn't exist in single-threaded JS. There's no second thread that could simultaneously be blocked trying to acquire the same lock, and a plain `Set` isn't lock-guarded at all. The Rust code needs this maneuver only because `tokio` genuinely runs tasks on multiple OS threads at once.

```
// idiomatic TS equivalent – this code does not exist in jcode
async function shutdown(scope: RuntimeTaskScope) {
  // No deadlock risk to design around here: JS has no second thread that
  // could be mid-way through acquiring a lock this code holds.
  const inFlight = [...scope["tasks"]];
  scope["tasks"].clear();
  await Promise.allSettled(inFlight);
}
```

Cancellation is cooperative, not preemptive

Both the accept loops and the per-connection handlers race the actual work against the cancellation token with `tokio::select!`:

```
// jcode: crates/jcode-app-core/src/server/runtime.rs:376–379
tokio::pin!(client);
tokio::select! {
  result = &mut client => Some(result),
  _ = cancellation.cancelled() => None,
}
```

This is the load-bearing idea for the whole chapter: **cancellation here is a request, not a kill switch**. Nothing forcibly stops the running task — it keeps going until it reaches a point where it's `.awaiting` something, racing that against "has cancellation fired." This is exactly the mental model behind `AbortSignal` in JS: code has to check `signal.aborted` itself, or hand the signal to something abort-aware like `fetch(url, { signal })`. Nothing yanks control away from code that's mid-execution.

```
// idiomatic TS equivalent – this code does not exist in jcode
async function runConnection(signal: AbortSignal, client:
    Promise<Result>) {
    return Promise.race([
        client.then((result) => ({ kind: "done" as const, result })),
        new Promise<{ kind: "cancelled" }>((resolve) => {
            signal.addEventListener("abort", () => resolve({ kind: "cancelled"
                })), {
                once: true,
            });
        }),
    ]);
}
```

Two spawn styles show up side by side in the file, worth a one-line note so it doesn't look like a rule the code doesn't actually enforce: the long-lived accept loops (`runtime.rs:156–185, 187–219`) are spawned directly with `tokio::spawn` and tracked by the caller's own returned handle, while every per-connection handler goes through `self.tasks.spawn(...)` so the scope can track and reap it. One accept loop (`spawn_gateway_accept_loop, runtime.rs:221–250`) is registered through the scope too — so it's not a strict "loops vs. connections" split, just two available tools used where each made sense.

Part B: InterruptSignal — cancelling an agent turn fast

The problem

Separate from server shutdown, jcode needs a way for a user hitting Esc/Ctrl+C to cancel an in-flight agent turn — stop streaming, stop a running tool — as fast as possible. Two failure modes to avoid: making the agent loop constantly ask "was I cancelled?" in a tight spin loop (wastes CPU), or missing a cancel that arrives at an inconvenient instant (a real bug users would feel as "my Esc didn't do anything"). `InterruptSignal` is built to satisfy both — cheap to check synchronously from a hot loop, and awaitable asynchronously without busy-waiting.

Its own doc comment states the design directly:

```
"Async-aware interrupt signal that combines AtomicBool (sync read) with
tokio::Notify (async wake). Eliminates spin-loops during tool execution."
— crates/jcode-agent-runtime/src/lib.rs:30–31
```

```
// jcode: crates/jcode-agent-runtime/src/lib.rs:32-40
#[derive(Clone)]
pub struct InterruptSignal {
    flag: Arc<std::sync::atomic::AtomicBool>,
    epoch: Arc<std::sync::atomic::AtomicU64>,
    notify: Arc<tokio::sync::Notify>,
}
```

An `AtomicBool` is a boolean that many threads can safely read and write without a lock. `tokio::sync::Notify` is async code's version of a condition variable — tasks can `.await` a call to `notified()` and get woken up when someone calls `notify_waiters()`. Together they give two ways to check the same signal: a synchronous, lock-free `is_set()` for hot paths, and an async, sleep-until-woken `notified()` for code that would rather not poll.

```
// jcode: crates/jcode-agent-runtime/src/lib.rs:51-55
pub fn fire(&self) {
    self.epoch.fetch_add(1, std::sync::atomic::Ordering::SeqCst);
    self.flag.store(true, std::sync::atomic::Ordering::SeqCst);
    self.notify.notify_waiters();
}
```

`fire()` always does all three steps together, so it doesn't matter whether the eventual observer is polling synchronously or waiting asynchronously — both see a consistent cancel.

The interesting part: why there's a third field

The `epoch` field is the one piece of this file worth slowing down for. Its own doc comment:

```
"Monotonic fire counter. Lets owners of a timed/deferred reset detect that
a newer fire landed in the meantime and skip the reset instead of erasing a
cancel the target has not observed yet (issue #428)." — crates/jcode-
agent-runtime/src/lib.rs:35-37
```

Concrete scenario: imagine something schedules "auto-clear this cancel in 500ms" right after firing it. If the user presses `Esc` *again* before those 500ms are up, a naive `reset()` would wipe out both the second, newer cancel *and* the auto-clear's own bookkeeping — the agent would silently un-cancel itself out from

under a user who had just cancelled it a second time. `reset_if_epoch` closes that gap: the resetter records which "version" of the signal it means to clear, and only actually clears the flag if no newer `fire()` happened since.

```
// jcode: crates/jcode-agent-runtime/src/lib.rs:78-90
pub fn reset_if_epoch(&self, epoch: u64) -> bool {
    if self.epoch.load(std::sync::atomic::Ordering::SeqCst) != epoch {
        return false;
    }
    self.flag.store(false, std::sync::atomic::Ordering::SeqCst);
    if self.epoch.load(std::sync::atomic::Ordering::SeqCst) != epoch {
        // A newer fire raced with the reset; restore it.
        self.flag.store(true, std::sync::atomic::Ordering::SeqCst);
        self.notify.notify_waiters();
        return false;
    }
    true
}
```

Notice the double-check: it reads the epoch, clears the flag, then reads the epoch *again* — because a new `fire()` could land in the gap between those two steps, on another thread, at any moment. If that happens, the function puts the flag (and the notification) back rather than letting a cancel get silently lost.

There's a companion race guarded in `notified()`:

```
// jcode: crates/jcode-agent-runtime/src/lib.rs:92-106
pub async fn notified(&self) {
    let mut notified = std::pin::pin!(self.notify.notified());
    notified.as_mut().enable();
    if self.is_set() {
        return;
    }
    notified.await;
}
```

The explicit `.enable()` registers this waiter with `Notify` *before* checking the flag, specifically so a `fire()` landing in the narrow window between "check the flag" and "start waiting" isn't missed. The file's own comments trace this back to a real, named bug ("issue #428"), and one of its tests hammers the race 2,000 times on a real multi-threaded runtime to catch it probabilistically — this is documentation of a bug that actually happened, not a hypothetical.

Why this is one of the best Rust-vs-TS contrasts in the whole

codebase: the entire epoch/enable-before-check dance exists to close races that can only happen because Rust code genuinely runs on multiple OS threads at once. In single-threaded JS, there is no possible interleaving between "I checked `signal.aborted`" and "someone else called `.abort()`" — the event loop guarantees one completes before the other's code runs at all. `AbortController` doesn't need an epoch counter or an "enable before check" step because the execution model makes the race physically impossible. This is a case where the honest TS equivalent is genuinely *simpler* than the Rust — not a simplification for teaching purposes, but a real reflection of what single-threadedness buys you.

```
// idiomatic TS equivalent – this code does not exist in jcode
class InterruptSignal {
  private fired = false;
  private emitter = new EventTarget();

  fire(): void {
    this.fired = true;
    this.emitter.dispatchEvent(new Event("fire"));
  }

  isSet(): boolean {
    return this.fired;
  }

  reset(): void {
    // No epoch/generation counter needed: within one turn of the event
    // loop, nothing else can have called fire() between two statements.
    this.fired = false;
  }

  notified(): Promise<void> {
    if (this.fired) return Promise.resolve();
    return new Promise((resolve) => {
      this.emitter.addEventListener("fire", () => resolve(), { once: true
    });
  });
}
```

How the two primitives relate

`RuntimeTaskScope`'s `CancellationToken` and `InterruptSignal` are two separate, differently-shaped primitives, not one wrapping the other. `CancellationToken` comes off the shelf from `tokio-util` and shuts down whole server tasks with built-in parent/child propagation, but it's one-shot — once cancelled, it stays

cancelled; there's no "reset." That's exactly why the agent turn needed its own hand-rolled primitive: `InterruptSignal`'s epoch-guarded reset lets a cancel be cleared and re-armed for the next turn, which `CancellationToken` simply has no concept of.

An honest gap, not filled in: the Phase 2 research behind this chapter traced `InterruptSignal`'s own file thoroughly, but did not trace the actual call sites where an Esc/Ctrl+C keypress calls `.fire()`, or where the agent's turn loop calls `.notified()/.is_set()`. The doc comments state the intent clearly enough ("eliminates spin-loops during tool execution," a reference to "agent stream loop, tool-wait select" in the `notified()` comment) but that's a statement of intent, not a confirmed call path — so this chapter stops at "this is what the primitive does and why it's shaped this way," not "here's the exact line where Esc reaches it."

Takeaways

- A spawned task with no owner is a task nobody can stop. `RuntimeTaskScope` exists purely to make sure every background task has an owner, at all times, until it's done.
 - Cancellation in tokio (and in JS's `AbortSignal`) is cooperative: it's a flag a task has to notice, not a kill switch. Both models share this limitation.
 - The deadlock `RuntimeTaskScope::shutdown` dodges — holding a lock across an `await` — is a genuinely Rust-shaped hazard. It doesn't have a JS analogue, because JS has no second thread to race against.
 - `InterruptSignal`'s epoch counter is solving a true-concurrency problem. It's one of the few places in this codebase where the idiomatic TS translation is meaningfully *simpler* than the Rust original, and that gap is itself the lesson.
-

Chapter 5 — Shared State Without Locks-as-a-Crutch

jcode's own design documentation makes a bold claim about how agents in a swarm coordinate:

"The system is optimistic by default (no locks)... Coordination happens via DM or channel, not through the coordinator." —
docs/SWARM_ARCHITECTURE.md:307–311

Then you open the Rust source and find `RwLock` and `Mutex` wrapped around almost every shared data structure in the server. Is the design doc lying?

No — and untangling *why not* is one of the more useful things a non-Rust reader can take from this codebase, because the confusion is common and the resolution is simple once you see it: **"no locks" and "uses locks" are true statements about two different layers of the same system.**

Two senses of "no locks"

Layer 1 — how agents coordinate with each other. When two agents in a swarm might both want to touch the same file, or claim the same piece of work, jcode does not make one agent block on a mutex owned by the server before it's allowed to act. There's no "acquire the file lock, then edit" step an agent has to go through. If a conflict happens, it gets resolved *after the fact* — through agents communicating (a completion report, a direct message, a file-touch notification) — not *prevented up front* by mutual exclusion. This is the layer the design doc is describing, and at this layer the claim is accurate: there is no lock an agent waits on to act.

Layer 2 — how the server keeps its own bookkeeping consistent.

Underneath all of that, the server itself is an ordinary multi-threaded program. It has one task running per client connection, one per background job, and so on — all genuinely running at once, on different OS threads, all needing to read and write shared in-memory data like "who are the current swarm members" or "what does the current plan look like." For that purely internal problem — not agent-to-

agent coordination, just keeping a shared `HashMap` from being corrupted by two threads writing to it simultaneously — the implementation reaches for the ordinary, correct tool: a lock around the shared structure. This isn't optional the way Layer 1's locklessness is a design choice; it's the baseline requirement of any correct multi-threaded program.

The sentence worth keeping in your head for the rest of this chapter: *"no locks" is a claim about how agents coordinate work with each other, not a claim about how the server implements its own shared memory.*

Layer 2, made concrete

The clearest example is `SwarmState` — the registry-of-registries that most of the server's swarm logic reads and writes:

```
// jcode: crates/jcode-app-core/src/server/state.rs:106-113
/// Shared ownership of the core persisted swarm coordination state.
#[derive(Clone)]
pub struct SwarmState {
    pub members: Arc<RwLock<HashMap<String, SwarmMember>>>,
    pub swarms_by_id: Arc<RwLock<HashMap<String, HashSet<String>>>>,
    pub plans: Arc<RwLock<HashMap<String, VersionedPlan>>>,
    pub coordinators: Arc<RwLock<HashMap<String, String>>>,
}
```

Four independently-locked hash maps: member records, a swarm-to-members index, plan storage, and coordinator assignments. `Arc` (atomic reference count) lets many owners hold a handle to the *same* underlying data rather than each getting a private copy — cloning `SwarmState` just clones four pointers, not the maps behind them. `RwLock` lets many concurrent *readers* in at once, or exactly one *writer* — unlike a plain mutex, which would make even simultaneous readers wait on each other. That distinction matters here specifically because status lookups vastly outnumber status changes.

None of this machinery has a reason to exist in single-threaded JS. A plain `Map` is only ever touched by one turn of the event loop at a time — there's no possibility of two pieces of code reading and writing it *simultaneously*, so there's nothing for a lock to protect against. (The one corner of JS where this stops being true is `worker_threads` combined with `SharedArrayBuffer` and `Atomics` — real shared

memory across real OS threads — which almost nobody reaches for.) The `Arc<RwLock<...>>` you see all over jcode's server code isn't over-engineering. It's the ordinary, unavoidable cost of a program that truly runs many threads at once.

```
// idiomatic TS equivalent — this code does not exist in jcode
// No lock needed: within one turn of Node's event loop, nothing else
// can be mid-write to this map while your code runs.
class SwarmState {
  members = new Map<string, SwarmMember>();
  swarmsById = new Map<string, Set<string>>>();
  plans = new Map<string, VersionedPlan>();
  coordinators = new Map<string, string>();
}
```

A real bug that a *narrower* lock fixed

It would be easy to conclude "locks are annoying, minimize them." jcode's own history argues for something more specific: when a lock gets in the way, the fix is usually a smaller, more targeted lock — not the absence of one. `state.rs` carries a small process-global registry that exists because of exactly this lesson:

```
// jcode: crates/jcode-app-core/src/server/state.rs:15-31
/// Process-global registry mapping session id -> background-tool signal.
///
/// The background-tool ("move tool to background", Alt+B/Ctrl+B) signal
/// lives on
/// the `Agent`, so a `SessionControlHandle` can normally only obtain it
/// by
/// locking the agent mutex. When a turn is busy (e.g. running
/// `await_members`),
/// `refresh_session_control_handle` falls back to a lock-free
/// `cancel_only`
/// handle that historically dropped the background signal entirely,
/// which made
/// Alt+B/Ctrl+B silently no-op (`BACKGROUND_TOOL_SIGNAL_FIRE
/// result=no_signal_handle`).
///
/// This registry is populated every time a full `SessionControlHandle`
/// is built
/// (which always has both the session id and the correct signal), so the
/// lock-free fallback can still fire the background signal without the
/// agent
/// lock. Entries are keyed by session id; renames/removals reuse
/// [`rename_background_tool_signal`]/[`remove_background_tool_signal`]
/// alongside
/// the existing shutdown-signal lifecycle.
static BACKGROUND_TOOL_SIGNALS: LazyLock<StdMutex<HashMap<String,
  InterruptSignal>>> =
  LazyLock::new(|| StdMutex::new(HashMap::new()));
```

The story in plain English: a hotkey ("move this running tool to the background") used to silently do nothing while an agent was busy. The *only* way to reach the signal it needed to fire was through the agent's own lock — and that lock was already held by the busy turn, so the fallback path just gave up and dropped the request. The fix wasn't to remove the lock. It was the opposite: add a second, small, independently locked side-registry that duplicates just the one signal handle the fallback path actually needs, so that path can reach it without ever touching the busy agent's lock.

This is the practical version of "optimistic, no locks" you'll actually find in the implementation: not zero locks, but *narrowing what sits behind each lock* so no single lock becomes a bottleneck other code paths get stuck behind.

```
// idiomatic TS equivalent – this code does not exist in jcode
// No lock needed in JS, but the *pattern* still ports: a small auxiliary
// side-index kept next to the main data, so a code path that can't touch
// the main structure right now can still reach the one thing it needs.
const backgroundToolSignals = new Map<string, InterruptSignal>();

function registerBackgroundToolSignal(sessionId: string, signal:
  InterruptSignal) {
  backgroundToolSignals.set(sessionId, signal);
}

function backgroundToolSignalForSession(sessionId: string):
  InterruptSignal | undefined {
  return backgroundToolSignals.get(sessionId);
}
```

The same shape again: lazy singletons

One more recurring pattern worth naming before moving to debouncing: a process-wide singleton built with a one-time-initializing cell.

```
// jcode: crates/jcode-base/src/bus.rs:499-502
pub fn global() -> &'static Bus {
  static INSTANCE: OnceLock<Bus> = OnceLock::new();
  INSTANCE.get_or_init(Bus::new)
}
```

OnceLock initializes exactly once, on first use, and is read-only after that — no ongoing locking cost. The same shape shows up again for the debounce bookkeeping later in this chapter (swarm.rs:108–113).

```
// idiomatic TS equivalent – this code does not exist in jcode
let instance: Bus | undefined;
function getBus(): Bus {
  if (!instance) instance = new Bus();
  return instance;
}
```

Part B: debounce instead of locking harder

There's a second instinct worth its own section, because it recurs independently across the codebase in a way that's genuinely interesting: when many concurrent writers are hammering some shared state and you want to tell clients about every change, the *tempting* fix is either a bigger lock (serialize everyone) or a message per change (correct, but wasteful — fifty status changes in one second doesn't need fifty near-identical broadcasts). jcode's actual answer, found independently at least three times in the server codebase, is neither: keep the lock small and short-held, and instead **coalesce a burst of rapid changes into one delayed, deduplicated broadcast**.

Occurrence 1 — swarm status broadcasts

The concrete cost that motivates this is stated directly in the source:

"How long terminal members stay in live SwarmStatus broadcasts... re-sending hundreds of long-finished members to every attached client on every status change dominates broadcast payloads (measured ~240 KB of member JSON resident per client with ~700 mostly-stopped members)."
— crates/jcode-app-core/src/server/swarm.rs:94–100

The debounce bookkeeping is a tiny struct behind its own small lock:

```

// jcode: crates/jcode-app-core/src/server/swarm.rs:87-88, 102-113
const DEFAULT_SWARM_STATUS_DEBOUNCE_MEMBER_THRESHOLD: usize = 2;
const DEFAULT_SWARM_STATUS_DEBOUNCE_MS: u64 = 75;

#[derive(Default, Clone, Copy)]
struct PendingSwarmStatusBroadcast {
    scheduled: bool,
    dirty: bool,
}

fn pending_swarm_status_broadcasts()
-> &'static StdMutex<HashMap<String, PendingSwarmStatusBroadcast>> {
    static PENDING: OnceLock<StdMutex<HashMap<String,
        PendingSwarmStatusBroadcast>>> =
        OnceLock::new();
    PENDING.get_or_init(|| StdMutex::new(HashMap::new()))
}

```

And the coalescing logic itself:


```

// jcode: crates/jcode-app-core/src/server/swarm.rs:758-781 (trimmed from
// the
// full function at 742-812)
if session_ids.len() < swarm_status_debounce_member_threshold() {
    broadcast_swarm_status_now(session_ids, swarm_members).await;
    return;
}

let key = swarm_broadcast_key(swarm_id, swarm_members, swarms_by_id);
let should_spawn = {
    let mut pending = pending_swarm_status_broadcasts()
        .lock()
        .unwrap_or_else(|poisoned| poisoned.into_inner());
    let entry = pending.entry(key.clone()).or_default();
    if entry.scheduled {
        entry.dirty = true; // someone's already waiting to flush; just
            mark "there's more"
        false
    } else {
        entry.scheduled = true;
        entry.dirty = false;
        true // we're first - we own the flush
    }
};
if !should_spawn {
    return;
}

tokio::spawn(async move {
    loop {
        tokio::time::sleep(Duration::from_millis(swarm_status_debounce_ms())).await
        // ...re-fetch current member statuses and broadcast the fresh
        snapshot...
        let mut pending = pending_swarm_status_broadcasts().lock()...;
        let Some(entry) = pending.get_mut(&key) else { break; };
        if entry.dirty {
            entry.dirty = false;
            continue; // more changes arrived during the sleep - loop
            again
        }
        pending.remove(&key);
        break; // caught up - stop; the next call spawns fresh
    }
});

```

The mechanism, in plain English: below a small threshold (2 members by default), every status change broadcasts immediately — not worth debouncing a 1-2 member swarm. Above that threshold, the *first* caller in a burst becomes the one that spawns a "sleep, then flush" loop. Every other caller that arrives while that loop is already scheduled just flips a dirty flag and returns — no new task,

no lock held longer than a single hashmap insert. When the sleep elapses, the loop broadcasts the *current* state, not whatever state existed when any particular caller fired, and if dirty got set again during the sleep, it loops and sleeps again rather than stopping — so a sustained burst keeps getting coalesced instead of the debounce "wearing off" after one cycle.

```
// idiomatic TS equivalent — this code does not exist in jcode
type PendingEntry = { timer: ReturnType<typeof setTimeout>; dirty:
    boolean };
const pending = new Map<string, PendingEntry>();

function broadcastSwarmStatus(swarmId: string, members: Map<string,
    SwarmMember>) {
    const sessionIds = getSessionIdsFor(swarmId);
    if (sessionIds.length < DEBOUNCE_MEMBER_THRESHOLD) {
        broadcastNow(sessionIds, members);
        return;
    }

    const entry = pending.get(swarmId);
    if (entry) {
        entry.dirty = true; // already scheduled — just note there's more
        return;
    }

    const timer = setTimeout(function flush() {
        broadcastNow(getSessionIdsFor(swarmId), members);
        const current = pending.get(swarmId)!;
        if (current.dirty) {
            current.dirty = false;
            current.timer = setTimeout(flush, DEBOUNCE_MS);
        } else {
            pending.delete(swarmId);
        }
    }, DEBOUNCE_MS);

    pending.set(swarmId, { timer, dirty: false });
}
```

This one ports almost 1:1. Debouncing is fundamentally a timing pattern, not a threading one, so it's one of the few places in this chapter where the Rust and the idiomatic TS really do look like the same code in two languages.

Occurrence 2 — same idea, different bookkeeping shape

`Bus::publish_models_updated` (`crates/jcode-base/src/bus.rs:542-606`) solves the identical problem — coalesce a burst of "models updated" events into one delayed publish — but tracks it differently: instead of a `scheduled/dirty` pair, it keeps `last_published_at: Option<Instant>` and a `publish_pending: bool`. If enough time has passed since the last publish, it publishes immediately; if not, and nothing is already pending, it schedules exactly one delayed publish for whatever time remains in the window; if something is already pending, it's a no-op. Same goal, same family, independently reinvented with different bookkeeping — worth knowing this exists, not worth a second full code block.

Occurrence 3 — a different flavor: leading-edge rate limiting

The third instance wasn't in this chapter's original research map — it turned up from directly grepping the server crate for "debounce," which is worth mentioning because it's a good reminder that "the notes said two" is worth checking rather than trusting outright:

```
// jcode: crates/jcode-app-core/src/server/client_state.rs:25-53 (called
// at line 897)
const ATTACH_MODEL_PREFETCH_DEBOUNCE_SECS: u64 = 15;

static LAST_ATTACH_MODEL_PREFETCH: LazyLock<StdMutex<HashMap<String,
Instant>>> =
    LazyLock::new(|| StdMutex::new(HashMap::new()));

fn should_debounce_attach_model_prefetch(provider_name: &str) -> bool {
    let Ok(mut guard) = LAST_ATTACH_MODEL_PREFETCH.lock() else {
        return false;
    };
    let now = Instant::now();
    if let Some(last_run) = guard.get(provider_name)
        && now.duration_since(*last_run) <
        Duration::from_secs(ATTACH_MODEL_PREFETCH_DEBOUNCE_SECS)
    {
        return true;
    }
    guard.insert(provider_name.to_string(), now);
    false
}
```

This one is a genuinely different shape, worth distinguishing by name: the first two occurrences are **trailing-edge coalescing** — wait, collect changes, do one combined thing at the end of a quiet period. This one is **leading-edge rate**

limiting — do the work immediately the first time, then suppress repeats within a cooldown window ("don't do this again too soon," here used to skip a model-catalog prefetch for a provider that was already prefetched recently on client attach). Different shape, same underlying instinct: a small lock around a tiny side-map is enough, because what's being protected is a few bytes of bookkeeping, not the actual work.

```
// idiomatic TS equivalent — this code does not exist in jcode
const lastAttachModelPrefetch = new Map<string, number>();
const DEBOUNCE_MS = 15_000;

function shouldDebounceAttachModelPrefetch(providerName: string): boolean
{
  const now = Date.now();
  const lastRun = lastAttachModelPrefetch.get(providerName);
  if (lastRun !== undefined && now - lastRun < DEBOUNCE_MS) {
    return true;
  }
  lastAttachModelPrefetch.set(providerName, now);
  return false;
}
```

Takeaways

- "No locks" in jcode's design docs describes how agents coordinate with each other (by communicating, not by contending for a mutex) — not how the Rust server manages its own in-memory state, which uses ordinary locks because it's an ordinary multi-threaded program.
 - None of the lock machinery you see in the Rust source has a reason to exist in single-threaded JS — a plain Map does the same job, because there's no possibility of two pieces of code touching it at the same instant.
 - When a lock in this codebase caused a real problem, the fix was a smaller, more targeted lock around exactly what a blocked code path needed — not the removal of locking altogether.
 - The debounce pattern shows up at least three times, independently, in two distinct shapes: trailing-edge coalescing (wait, collect, flush once) and leading-edge rate limiting (do it now, suppress repeats). Both exist so the lock stays small; what gets smarter is *when* you act, not how hard you serialize access.
-

Chapter 6 — The Task DAG (Deep Mode)

A correction, made in public

Before we get to the code, a small confession about how this very chapter got written.

The first pass at mapping this codebase assumed the recursive task-graph engine — the thing that lets an agent break a task into sub-tasks, and those sub-tasks into further sub-tasks — lived in a crate called `jcode-swarm-core`. That assumption was wrong. `jcode-swarm-core` does contain the strings `expand_node`, `complete_node`, and `inject_gap` — but only inside functions that generate prompt *text* for the LLM (`append_deep_node_instructions`, `append_deep_gate_instructions`, `crates/jcode-swarm-core/src/lib.rs:390–433` and `449–499`). It doesn't implement the graph at all.

The real engine lives in `crates/jcode-plan/src/dag/` — `mod.rs`, `ops.rs`, `schedule.rs`. The mistake was caught only because a later pass opened the files directly instead of trusting the earlier summary, and re-derived every claim from the actual function bodies.

Hold onto that story. By the end of this chapter you'll see that `jcode` bakes the exact same discipline — don't trust a self-report, re-check the actual evidence — into its own task graph, as a first-class engine feature rather than a one-off writing habit.

The problem, in plain English

Say you let an LLM agent break a big task into an unbounded tree of sub-tasks, and those sub-tasks spawn further sub-tasks, and other agents (including itself) pick them up and work on them in parallel. Two things can go wrong:

1. **The tree gets structurally broken.** Cycles. Sub-tasks that reference work that doesn't exist. Two workers editing the same node at once.
2. **The tree gets shallow.** An agent says "done" without having actually explored the scope it was given — and nothing forces it to admit what it skipped.

jcode's answer is one data structure — a DAG (directed acyclic graph) of task nodes — plus a small, closed set of **validated mutation functions** that are the only legal way to change it. Nothing else is allowed to touch the node list directly:

```
// jcode: crates/jcode-plan/src/dag/mod.rs:541 — the node list is
    private,
// not `pub`. Every mutation has to go through the ops:: functions below.
nodes: Vec<TaskNode>
```

Think of it like a REST API that never lets a client write straight to a database table — every write goes through a handler that can say no. The four "endpoints" here line up with a task's actual lifecycle:

1. **seed** — lay down the initial task list.
2. **expand_node** — an agent decomposes a node it owns into children.
3. **complete_node** — an agent finishes a node with a typed report.
4. **inject_from_gate** — a reviewer node finds a gap and adds work instead of rubber-stamping.

Every one of these can *fail* with a structured, actionable error instead of silently corrupting the graph. We'll get to what that looks like.

One engine, two presets: Mode

```
// jcode: crates/jcode-plan/src/dag/mod.rs:37-49
pub enum Mode {
    Deep,
    Light,
}
impl Mode {
    pub fn requires_gates(self) -> bool {
        matches!(self, Mode::Deep)
    }
}
```

The doc comment above this enum is worth quoting verbatim, because it's the single best "why" sentence in the whole engine:

"One engine, two presets... The data model, scheduler, and dataflow are identical; the mode only controls whether the rigor machinery (mandatory gates + strict artifact validation) is engaged."

Deep and Light aren't two different schedulers, or two different graph shapes. It's the *same* code, gated behind one predicate function (`requires_gates()`), consulted at exactly the two points that matter: does `expand_node` auto-insert a reviewer, and does `complete_node` enforce strict validation on the finished work. That's a clean pattern worth stealing on its own, independent of task graphs: a feature flag reduced to one function, checked at its call sites, instead of `if mode == "deep"` scattered through the codebase.

```
// idiomatic TS equivalent – this code does not exist in jcode
type Mode = "deep" | "light";

function requiresGates(mode: Mode): boolean {
  return mode === "deep";
}
```

One note before we go further: this "mode" has nothing to do with whether a spawned worker gets its own terminal window (that's a separate setting, `SwarmSpawnMode`, covered in Chapter 2 and Chapter 8's glossary). Two different axes, same word — don't conflate them.

Where a node came from: `NodeOrigin`

```
// jcode: crates/jcode-plan/src/dag/mod.rs:58–67
// Seed | Expand | Gap | Gate
```

Seed nodes are the first agent's rough draft. Everything else — `Expand`, `Gap`, `Gate` — is growth the machinery generated afterward. The doc comment explains the point of tagging this at all: "a plan that never outgrew its seed is visibly under-explored." `NodeOrigin` doesn't drive any control flow inside the engine — it's pure provenance, there so a status report can say "6 of 9 nodes in this graph were never part of the original draft," which is a useful signal that real decomposition happened instead of an agent skimming the surface and calling it done.

```
// idiomatic TS equivalent – this code does not exist in jcode
type NodeOrigin = "seed" | "expand" | "gap" | "gate";
```

What kind of node, and who reviews it: NodeKind and gate_kind()

```
// jcode: crates/jcode-plan/src/dag/mod.rs:72-102
pub enum NodeKind {
    Explore,
    Implement,
    Verify,
    Fix,
    Synthesize,
    Critique,
}

impl NodeKind {
    pub fn is_gate_kind(self) -> bool {
        matches!(self, NodeKind::Critique | NodeKind::Verify)
    }

    pub fn gate_kind(self) -> NodeKind {
        match self {
            NodeKind::Implement | NodeKind::Fix => NodeKind::Verify,
            _ => NodeKind::Critique,
        }
    }
}
```

gate_kind() is the rule that decides who reviews a piece of work. Code-shaped work — Implement, Fix — gets a Verify gate: something with an objective pass/fail signal, like "did it build, did the tests pass." Everything else — Explore, Synthesize, even Critique reviewing itself — falls through to a Critique gate: an adversarial second read hunting for gaps, because "did you explore enough" has no compiler to check it. One function collapses a six-value enum down to a binary review-strategy choice, instead of special-casing each variant at every call site that needs to know how to review it.

```
// idiomatic TS equivalent – this code does not exist in jcode
type NodeKind = "explore" | "implement" | "verify" | "fix" | "synthesize"
  | "critique";

function isGateKind(kind: NodeKind): boolean {
    return kind === "critique" || kind === "verify";
}

function gateKindFor(kind: NodeKind): NodeKind {
    if (kind === "implement" || kind === "fix") return "verify";
    return "critique";
}
```


Four states, not five: NodeStatus

```
// jcode: crates/jcode-plan/src/dag/mod.rs:107-116  
// Queued | Running | Done | Failed
```

Notice what's missing: Blocked. The doc comment says why, and it's a lesson worth generalizing: "'Blocked' is intentionally not stored: it is computed from dependency state by the scheduler, so there is a single source of truth." A node is blocked exactly when it's Queued and some dependency isn't Done yet — that's fully derivable from data already on hand, so storing it as a fifth enum variant would just create a second copy of the truth that could quietly drift out of sync with the real one.

```
// idiomatic TS equivalent – this code does not exist in jcode  
type NodeStatus = "queued" | "running" | "done" | "failed";  
  
// "blocked" is never stored – it's a computed view:  
function isBlocked(node: Node, graph: Map<string, Node>): boolean {  
  return (  
    node.status === "queued" &&  
    node.dependsOn.some((id) => graph.get(id)?.status !== "done")  
  );  
}
```

Don't store what you can compute — the same rule that keeps a derived `fullName` getter off a database row.

Reading confidence out of free text: ConfidenceLevel::parse

When a node finishes, the agent that did the work self-reports how confident it is. That confidence has to come out of free-form LLM text, not a clean radio button, which makes parsing it a small adversarial problem in its own right.

```
// jcode: crates/jcode-plan/src/dag/mod.rs:129-133 (enum), 141-204  
(parse)  
pub enum ConfidenceLevel {  
  Low,  
  Medium,  
  High,  
}
```

The doc comment states the stakes plainly:

"Confidence is the breadth signal of the task graph: a node completed at `ConfidenceLevel::Low` is an admission that its scope was not adequately covered, so the machinery treats it like `what_i_did_not_check` — gates are pointed at low-confidence siblings and (in deep mode) cannot pass while such a sibling is unaddressed."

parse is genuinely careful about a real bug class. It checks **negation phrases first** ("not confident," "unsure," "uncertain") before it looks for word-rungs like "high." The reason: the substring "confident" appears *inside* "not confident", so a naive "does this text contain the word high-confidence" check would read "not confident" as a High rating — exactly backwards, and exactly the kind of self-report the gate mechanism exists to catch. Only after the negation check does it look for word rungs (checking "low" before "high", so a hedge like "low-to-high" resolves pessimistically), and only after that does it fall back to parsing numbers — disambiguating a bare "0.9" as a probability from a bare "1" as a 1-of-10 score.

```
// idiomatic TS equivalent — this code does not exist in jcode
type ConfidenceLevel = "low" | "medium" | "high";

function parseConfidence(raw: string): ConfidenceLevel | null {
  const text = raw.trim().toLowerCase();
  if (!text) return null;

  // Negation phrases must be checked BEFORE word-rung substrings —
  // "not confident" contains "confident," which would otherwise
  // false-positive as a High rating.
  const negations = ["not confident", "unsure", "uncertain", "not sure"];
  if (negations.some((phrase) => text.includes(phrase))) return "low";

  if (text.includes("low")) return "low";
  if (text.includes("high")) return "high";
  if (text.includes("medium") || text.includes("moderate")) return
    "medium";

  // (jcode also falls back to numeric parsing here — 1-10 scores,
  // fractions, percentages — omitted from this sketch for brevity.)
  return null;
}
```

The ordering is the entire lesson: run the negation check first, or a confident-sounding piece of text about *not* being confident silently erases the exact signal the rest of the engine depends on.

The four legal ways to touch the graph

Every mutation function follows the same skeleton: check who's allowed to do this and whether the graph is in the right state, validate the proposed change, then commit. Two of the three we'll look at go further and use a **stage-on-a-clone** pattern: build the new graph on a full clone, run a cycle check against the clone, and only overwrite the real graph if the clone is still acyclic.

expand_node — decomposing a node into children

```
// jcode: crates/jcode-plan/src/dag/ops.rs:227-367
pub fn expand_node(/* graph, node_id, actor, children */) ->
    Result<ExpandOutcome, DagError> {
    // 1. ownership + state guard (233-260): reject unless `actor` owns
    //    the node, reject gates decomposing themselves, reject a node
    //    that's already expanded or not currently Running.
    // 2. validate every child id/dependency (264-284): no id collisions,
    //    every depends_on must resolve to a real node.
    let mut staged = graph.clone(); // ops.rs:287
    // 3. insert children, parented to this node, tagged
    //    NodeOrigin::Expand
    // 4. deep mode only: auto-insert a gate, kind =
    //    parent_kind.gate_kind()
    // 5. flip the parent into a composite join: expanded = true,
    //    status = Queued, append child ids AND the gate id to its own
    //    depends_on
    let cycle = staged.cycle_nodes();
    if !cycle.is_empty() {
        return Err(DagError::WouldCreateCycle(cycle)); // ops.rs:361-364
    }
    *graph = staged; // ops.rs:365
    // ...
}
```

Two details worth slowing down for.

First, the **stage-then-commit** shape: nothing mutates the live graph until a full copy of the proposed result has already passed the cycle check. If the check fails, the clone is just discarded — the real graph never saw the bad state. It's the same shape as a database transaction, minus the database.

Second — and this one is a genuinely non-obvious, verified nuance — in deep mode the parent node ends up depending on *both* its new children *and* the auto-inserted gate, even though the gate itself already depends on all the children. That looks redundant (if the gate is done, the children must already be done too),

but it isn't: the scheduler's dependency check only cares whether everything is *transitively* done, while the step that assembles a worker's input reads only a node's *direct* dependencies. If the child edges were dropped once the gate existed, the eventual synthesis step would never receive the children's actual output when it wakes back up. The duplicate edges are load-bearing, not an oversight.

```

// idiomatic TS equivalent – this code does not exist in jcode
type Result<T> = { ok: true; value: T } | { ok: false; error: string };

interface Node {
  id: string;
  kind: NodeKind;
  status: NodeStatus;
  owner: string | null;
  parent: string | null;
  dependsOn: string[];
  expanded: boolean;
  isGate: boolean;
}

function expandNode(
  graph: Map<string, Node>,
  nodeId: string,
  actor: string,
  children: { id: string; kind: NodeKind; dependsOn?: string[] }[],
): Result<{ childIds: string[]; gateId: string | null }> {
  const node = graph.get(nodeId);
  if (!node) return { ok: false, error: "unknown node" };
  if (node.owner !== actor) return { ok: false, error: "not owner" };
  if (node.isGate) return { ok: false, error: "gate cannot expand itself" };

  if (node.status !== "running" || node.expanded) {
    return { ok: false, error: "invalid state" };
  }
  if (children.length === 0) return { ok: false, error: "empty children" };

  for (const child of children) {
    if (graph.has(child.id)) return { ok: false, error: `duplicate id ${child.id}` };
    for (const dep of child.dependsOn ?? []) {
      if (!graph.has(dep) && !children.some((c) => c.id === dep)) {
        return { ok: false, error: `unknown dependency ${dep}` };
      }
    }
  }

  // Stage on a copy; never touch the real graph until it passes.
  const staged = new Map(graph);
  const childIds = children.map((c) => c.id);
  for (const child of children) {
    staged.set(child.id, {
      id: child.id,
      kind: child.kind,
      status: "queued",
      owner: null,
      parent: nodeId,
      dependsOn: child.dependsOn ?? [],
    });
  }
}

```

```

        expanded: false,
        isGate: false,
    });
}

let gateId: string | null = null;
// (deep-mode-only gate insertion omitted for brevity – same shape:
// insert one more node depending on all childIds, tagged isGate: true)

const parent = { ...node, expanded: true, status: "queued" as const,
    owner: null };
parent.dependsOn = [...parent.dependsOn, ...childIds, ...(gateId ?
    [gateId] : [])];
staged.set(nodeId, parent);

if (hasCycle(staged)) {
    return { ok: false, error: "would create a cycle" };
}
for (const [id, n] of staged) graph.set(id, n); // commit
return { ok: true, value: { childIds, gateId } };
}

function hasCycle(graph: Map<string, Node>): boolean {
    const WHITE = 0, GRAY = 1, BLACK = 2;
    const color = new Map<string, number>();
    const visit = (id: string): boolean => {
        color.set(id, GRAY);
        for (const dep of graph.get(id)?.dependsOn ?? []) {
            const c = color.get(dep) ?? WHITE;
            if (c === GRAY) return true; // back edge -> cycle
            if (c === WHITE && visit(dep)) return true;
        }
        color.set(id, BLACK);
        return false;
    };
    for (const id of graph.keys()) {
        if ((color.get(id) ?? WHITE) === WHITE && visit(id)) return true;
    }
    return false;
}

```

complete_node — finishing a node with a typed report

```
// jcode: crates/jcode-plan/src/dag/ops.rs:377-411
// 1. ownership + state check (must be Running)
// 2. validate_artifact(mode, node_id, is_gate, &artifact) — ops.rs:400
//    deep mode, non-gate nodes: reject empty `findings`, reject an empty
//    `what_i_did_not_check` list, reject a `confidence` field that
//    doesn't
//    parse to a ConfidenceLevel
// 3. if this node is a gate and mode requires gates:
//    validate_gate_pass(graph, node_id, &artifact) — ops.rs:402 (see
//    below)
// 4. on success: node.status = Done; node.output = Some(artifact);
```

No stage-then-commit dance here — completing a node never changes an edge, so there's no cycle to guard against. It's a simpler shape: validate everything about the incoming artifact first, and only after every check passes does a single field get written. Same idea as a form-submit handler that refuses to touch application state until the whole payload has passed validation.

The interesting design decision is in step 2: **light mode and gate artifacts skip the strict checks entirely**. A gate's artifact is a pass/fail record about someone else's work, not a scope claim about its own — so it doesn't need to declare "what I did not check" the way a regular worker node does.

```

// idiomatic TS equivalent – this code does not exist in jcode
interface HandoffArtifact {
  findings: string;
  evidence: string[];
  whatIDidNotCheck: string[];
  confidence?: string;
}

function completeNode(
  graph: Map<string, Node>,
  nodeId: string,
  actor: string,
  mode: Mode,
  artifact: HandoffArtifact,
): Result<void> {
  const node = graph.get(nodeId);
  if (!node) return { ok: false, error: "unknown node" };
  if (node.owner !== actor) return { ok: false, error: "not owner" };
  if (node.status !== "running") return { ok: false, error: "invalid
    state" };

  if (mode === "deep" && !node.isGate) {
    if (!artifact.findings) return { ok: false, error: "thin artifact: no
      findings" };
    if (artifact.whatIDidNotCheck.length === 0) {
      return { ok: false, error: "thin artifact: must state what was not
        checked" };
    }
    if (parseConfidence(artifact.confidence ?? "") === null) {
      return { ok: false, error: "thin artifact: unparseable confidence"
        };
    }
  }
}

// (gate nodes in deep mode additionally run validateGatePass here –
// see the anti-hallucination section below)

graph.set(nodeId, { ...node, status: "done" });
return { ok: true, value: undefined };
}

```


inject_from_gate — a reviewer that doesn't rubber-stamp

```
// jcode: crates/jcode-plan/src/dag/ops.rs:444-540
// 1. must be a gate, must be Running, must supply >= 1 new node
// 2. validate new ids/deps, same as expand_node
// 3. stage-then-commit on a clone, same as expand_node
// 4. new nodes are parented to the GATE'S PARENT (ops.rs:505,
//     NodeOrigin::Gap) — they become siblings of whatever the gate
//     was auditing, not children of the gate
// 5. the gate re-queues itself (ops.rs:508-519): status = Queued,
//     owner = None, and the new node ids are appended to the gate's
//     own depends_on — so the gate re-runs once the gap work is done
```

This is `expand_node`'s twin, with one different rule: instead of a review node finding a problem and walking away, it files new work *and puts itself back in the queue* to re-check once that work lands. It's the same shape as a CI pipeline where a failing lint step doesn't just fail the build — it opens tickets and re-schedules itself to re-run once they close.

```
// idiomatic TS equivalent – this code does not exist in jcode
function injectFromGate(
  graph: Map<string, Node>,
  gateId: string,
  actor: string,
  newNodes: { id: string; kind: NodeKind; dependsOn?: string[] }[],
): Result<{ newIds: string[] }> {
  const gate = graph.get(gateId);
  if (!gate) return { ok: false, error: "unknown node" };
  if (!gate.isGate) return { ok: false, error: "not a gate" };
  if (gate.owner !== actor) return { ok: false, error: "not owner" };
  if (gate.status !== "running") return { ok: false, error: "invalid
    state" };
  if (newNodes.length === 0) return { ok: false, error: "no nodes
    supplied" };

  const staged = new Map(graph);
  const newIds = newNodes.map((n) => n.id);
  for (const n of newNodes) {
    staged.set(n.id, {
      id: n.id,
      kind: n.kind,
      status: "queued",
      owner: null,
      parent: gate.parent, // sibling of what the gate was auditing
      dependsOn: n.dependsOn ?? [],
      expanded: false,
      isGate: false,
    });
  }
  staged.set(gateId, {
    ...gate,
    status: "queued",
    owner: null,
    dependsOn: [...gate.dependsOn, ...newIds],
  });

  if (hasCycle(staged)) return { ok: false, error: "would create a cycle"
  };
  for (const [id, n] of staged) graph.set(id, n);
  return { ok: true, value: { newIds } };
}
```

The worked example: multimonitor support in scrollwm

docs/SWARM_TASK_GRAPH.md §9 walks through a scenario end to end — an agent exploring multimonitor support in a window manager — and it lines up exactly with the mechanics above. Here's the graph evolving over time:

- **T0 — seed.** The first agent lays down a skeleton, not an answer: one explore root, a critique gate, a synthesize node.
- **T1 — expand into facets.** The root decomposes into six sibling facets — geometry/layout, hotplug, DPI/scaling, focus/cursor, workspace map, existing-code touchpoints — all depending into the same critique gate. This is `expand_node`, called once.
- **T2 — fan-out.** The scheduler hands each queued facet with satisfied dependencies to a worker.
- **T3 — recursion.** One facet, hotplug, turns out to have its own deep scope. Its owning worker calls `expand_node` *on the node it owns* — the same function, called again, one level down. There is no separate "recursion" code path; it's the same validated mutation, applicable at any depth by whoever currently owns a node.
- **T4 — atomic facets finish; edges start carrying data.** Four of the six facets complete with typed artifacts. The critique gate is still blocked — it depends on the still-running hotplug subtree.
- **T5 — reduce.** The hotplug children finish; the same worker that expanded hotplug (recorded as its planner) wakes back up to synthesize one clean hotplug report. The hotplug composite closes.
- **T6 — the gate finds a gap.** The critique gate reads every facet's `what_i_did_not_check` list and notices nobody covered "fullscreen on one output" or "mixed refresh rate." It calls `inject_from_gate` with two new gap nodes and re-queues itself for a second pass. The top-level synthesize node stays blocked, transitively, through the gate.
- **T7 — gap nodes finish, re-critique passes, synthesis runs.** The final synthesis assembles every upstream artifact *by reference* — evidence strings pointing at file:line and commit references, not embedded content — into the final report.

The design doc's own closing line for this section is worth carrying into this chapter too: "The graph is never drafted once; it grows wherever depth or gaps are found and shrinks in attention as subtrees collapse into synthesized artifacts."

Gates as jcode's own anti-hallucination mechanism

This is the chapter's central exhibit: `validate_gate_pass` (`crates/jcode-plan/src/dag/ops.rs:789-878`) is the function that decides whether a reviewer's "looks good" is actually allowed to close the graph out. It runs three checks, in

this order:

1. **Stale scope.** Every node the gate is auditing must already be Done. If any are still pending, the pass is rejected outright — this guards against the gate racing ahead of work that got added after it started.
2. **Confidence debt.** Any audited node that self-reported `ConfidenceLevel::Low` must be explicitly named, by id, in the gate's own findings or open questions — or already patched via a prior `inject_from_gate` call. Critically, the gate's *own* `what_i_did_not_check` field does not count as addressing this. The doc comment is blunt about why: "declaring 'I did not check X' is the opposite of addressing X."
3. **Coverage debt.** Up to a cap of 20 audited nodes, the passing artifact must name *every* done node in scope — not just the shaky ones. "All good, no gaps" isn't allowed to pass silently over work it never even mentions. Past the cap, full enumeration relaxes, but anything that didn't self-report High confidence — medium, low, or unparseable — still has to be named.

Naming a node "by mentioning its id in text" sounds trivial until you notice a short id like "a" would match almost any English sentence under a naive substring check. jcode instead uses a hand-rolled word-boundary scanner (`mentions_node_id`, `ops.rs:593-628`) that treats characters like `-_.` as legal id characters but still requires a true boundary on either side — a small, real correctness bug avoided, not a triviality.

Every failure here comes back as a structured, actionable error — not a bare "validation failed." The error for unaddressed low confidence, for instance, names the exact two things the gate could do next: call `inject_from_gate` with follow-up nodes, or name the id in its own findings.

```

// idiomatic TS equivalent – this code does not exist in jcode
type GateFailure =
  | { ok: false; kind: "stale_scope"; pending: string[] }
  | { ok: false; kind: "unaddressed_low_confidence"; nodes: string[] }
  | { ok: false; kind: "uncovered_siblings"; nodes: string[] };

function validateGatePass(
  graph: Map<string, Node>,
  gateId: string,
  artifact: HandoffArtifact,
): { ok: true } | GateFailure {
  const gate = graph.get(gateId)!;
  const scope = gate.dependsOn.filter((id) => !graph.get(id)?.isGate);

  const pending = scope.filter((id) => graph.get(id)?.status !== "done");
  if (pending.length > 0) return { ok: false, kind: "stale_scope",
    pending };

  const lowConfidence = scope.filter(
    (id) => graph.get(id)?.output?.confidence === "low",
  );
  const unaddressed = lowConfidence.filter(
    (id) => !mentionsNodeId(artifact.findings, id),
  );
  if (unaddressed.length > 0) {
    return { ok: false, kind: "unaddressed_low_confidence", nodes:
      unaddressed };
  }

  const uncovered = scope.filter((id) =>
    !mentionsNodeId(artifact.findings, id));
  if (uncovered.length > 0) {
    return { ok: false, kind: "uncovered_siblings", nodes: uncovered };
  }

  return { ok: true };
}

function mentionsNodeId(text: string, id: string): boolean {
  // word-boundary aware, not a bare text.includes(id) –
  // a short id like "a" would otherwise match almost any sentence
  const pattern = new RegExp(`(^|^[^\\w.:-])${id.replace(/[\.*?^$\\{\}()|
    []\\]/g, "\\$&")}([\\w.:-]|$)`);
  return pattern.test(text);
}

```

A nice analogy: this is a CI merge gate that programmatically checks a reviewer's approval comment actually mentions every changed file above some risk threshold, instead of accepting a bare "LGTM."

The callback

Here's the honest parallel this chapter has been building toward. jcode does not let a task node close out on an agent's self-report that it's "done" and "fine." A second, adversarial node — the gate — is structurally required to re-examine the completed work, specifically hunting for what the first agent admitted it didn't check. And the *engine itself*, not just a prompt instructing it to be careful, refuses to let that gate rubber-stamp past an unaddressed low-confidence node or a sibling it never mentioned.

That's the same shape as the process that produced this handbook. A cite-and-verify pass re-opens every source file a draft claims something about, and a claim doesn't get accepted just because it's stated more confidently the second time — it gets accepted because someone (or something) went and checked. The correction at the top of this chapter wasn't a special case. It was the same mechanism, one level up, catching the same kind of mistake jcode's own gates exist to catch.

Chapter 7 — Communication Topology

If Chapter 3 was about running things in parallel, this chapter is about what agents actually say to each other once they're running: who can send a message to whom, who can read what about another agent's state, and how a piece of news travels from one part of the system to every place that cares about it.

One framing note before the code, because it changes how you should read everything below. jcode has two design docs describing this territory, and they're not equally current. `docs/SWARM_ARCHITECTURE.md` opens by describing itself as "largely implemented," but points to `docs/SWARM_TASK_GRAPH.md` for the newer, DAG-first model that supersedes its agent-first framing — and says its own communication section is mid-migration. `SWARM_TASK_GRAPH.md` §8a lays out a four-step plan to trim a chattier, chat-style communication surface down to something leaner:

1. **Done.** Completion artifacts flow to dependents automatically.
2. **Done.** Broadcasts are scoped to the sender's spawned subtree, not the whole swarm.
3. Migrate existing flows off channels and shared-context. (pending)
4. Deprecate, then remove, the redundant chat primitives. (pending)

So: what follows is "what's actually live today," described mostly in the older doc's vocabulary (DMs / broadcast / channels), with step 2 of that migration — subtree-scoped broadcast — already landed in code. Treat this as a system partway through simplifying itself, not a finished design.

DMs, subtree broadcast, and channels

All three routing shapes go through one function: `handle_comm_message` in `crates/jcode-app-core/src/server/client_comm_message.rs`. A single call picks its delivery shape based on which optional fields the caller filled in:

```
// jcode: crates/jcode-app-core/src/server/client_comm_message.rs:217-223
let scope = if resolved_to_session.is_some() {
    "dm"
} else if channel.is_some() {
    "channel"
} else {
    "broadcast"
};
```

to_session wins if present, then channel, and if neither is set it falls through to broadcast. This single function auto-routing among three delivery shapes is exactly the property SWARM_TASK_GRAPH.md §8a points to as evidence the surface has too many overlapping primitives — worth reading as the design doc's own retrospective critique of the code you're looking at, not an endorsement of it.

DMs. A DM target can be an exact session ID, or a swarm-unique "friendly name" resolved by resolve_dm_target_session(client_comm_message.rs:35-80). If a friendly name matches more than one session, that's a hard error naming every match — it doesn't silently pick one for you.

Subtree broadcast. This is the concrete implementation of migration step 2 above:

```
// jcode: crates/jcode-app-core/src/server/client_comm_message.rs:230-249
// Broadcast-style sends are subtree-scoped: a sender reaches only the
// agents it (transitively) spawned, via the report-back ancestry chain.
// The swarm coordinator keeps whole-swarm reach as an escape hatch.
// This prevents one agent from producing a member-cap-sized
// notification storm (see docs/SWARM_TASK_GRAPH.md section 8a).
let subtree_broadcast_targets: Vec<String> = {
    let members = swarm_members.read().await;
    let sender_is_coordinator = members
        .get(&from_session)
        .is_some_and(|member| member.role == "coordinator");
    swarm_session_ids
        .iter()
        .filter(|session_id| *session_id != &from_session)
        .filter(|session_id| {
            sender_is_coordinator
                || super::swarm_is_self_or_ancestor(&members,
                    &from_session, session_id)
        })
        .cloned()
        .collect()
};
```


A plain agent's "broadcast" only reaches sessions it transitively spawned — tracked via the report-back ancestry chain (Chapter 2's territory). Only the coordinator gets true whole-swarm reach, as a deliberate escape hatch. Notice the reused primitive: `swarm_is_self_or_ancestor` is the same authorization check used elsewhere in `swarm.rs` to decide who may stop a session — one function, two different jobs.

Also notice `member.role == "coordinator"` here compares against a bare string, not a typed enum. Hold that thought — it's the seed of the biggest finding in this chapter, in the last section below.

Channels. The least emphasized of the three, and the code shows it:

```
// jcode: crates/jcode-app-core/src/server/client_comm_message.rs:253-269
} else if let Some(ref channel_name) = channel {
    let subs = channel_subscriptions.read().await;
    let index = ChannelIndex {
        by_swarm_channel: subs.clone(),
        by_session: HashMap::new(),
    };
    let channel_members = index.members(&swarm_id, channel_name);
    if channel_members.is_empty() {
        // No subscribers: fall back to the subtree scope rather than
        // blasting the whole swarm.
        subtree_broadcast_targets.clone()
    } else {
        channel_members
            .into_iter()
            .filter(|session_id| session_id != &from_session)
            .collect()
    }
}
```

An empty channel silently falls back to the subtree-broadcast scope rather than erroring or delivering nothing — channels behave here as "a broadcast pre-filter that no-ops back to broadcast if unused." That's consistent with `SWARM_ARCHITECTURE.md`'s own framing of channels as discouraged in favor of DMs. It's also worth noting the code builds a brand-new `ChannelIndex` from the raw subscription map on every single call, rather than holding a persistent index — the actual long-lived state is just the underlying `channel_subscriptions` map; `ChannelIndex` is a one-shot lookup helper around it, not a registry in its own right.

Whichever of the three shapes resolves a target list, delivery itself goes through one shared helper, `fanout_session_event`, which per `SWARM_ARCHITECTURE.md` queues these as "soft interrupts" injected into a running agent at safe points. That specific injection-timing claim comes from the design doc — this chapter's source notes did not independently trace `queue_soft_interrupt_for_session`'s implementation, so treat "safe points" as the documented intent rather than something verified line by line here.

Three ways to read another agent's state

A swarm member can ask about another member in three different ways, each with a different cost and a different availability guarantee. All three live in `crates/jcode-app-core/src/server/comm_sync.rs`, and all three sit behind one shared gate: `ensure_same_swarm_access` (`comm_sync.rs:144–176`) — requester and target must share a `swarm_id` before any of what follows runs.

Status snapshot — `handle_comm_status`, `comm_sync.rs:249–324`. The cheapest tier, and designed to never block. The core snapshot is assembled entirely from the `swarm_members` registry — a data-structure lock (`Arc<RwLock<HashMap<...>>>`), not the per-agent mutex — plus file-touch and connection state. It only *opportunistically* reaches for the target agent's own lock, using `try_lock()`, to grab provider/model name; if that fails because the agent is busy, those two fields just come back empty rather than the call blocking or erroring.

Summary read — `handle_comm_summary`, `comm_sync.rs:194–243`. A step up in cost: it must lock the target agent (`try_lock`) to call `agent.get_tool_call_summaries(limit)` (default limit 10) — a short activity feed of recent tool calls. Unlike status, this tier does *not* degrade quietly if the lock fails; it returns an explicit error, "Session '{}' is busy; try summary again shortly", with a `retry_after_secs: 1` hint. Bounded and lightweight when it works, but it can fail transiently under load.

Full context read — `handle_comm_read_context`, `comm_sync.rs:326–382`. The expensive, gated tier: the entire message history via `agent.get_history()`. Two layers of protection beyond the other two tiers. First, `can_read_full_context` (`comm_sync.rs:178–192`) restricts this to the session itself or a session whose `role == "coordinator"` — again a bare-string comparison, not the typed role

enum — with an explicit error for anyone else: "Only the coordinator, worktree manager, or the target session may read full context. Use summary for lightweight access." Second, the same try-lock-or-explicit-busy-error pattern as the summary tier.

Same underlying data — an Agent behind a mutex, plus a lock-free metadata registry — exposed at three deliberately different cost points: always-available-but-shallow, cheap-but-can-fail-busy, and expensive-and-permission-gated. It's a clean design to hold onto as a pattern in its own right: when multiple callers need different amounts of detail about the same live object, give them different endpoints with different locking guarantees, rather than one endpoint that's either always slow or always risks blocking on a busy resource.

The global event bus

Separate from any of the routing above, jcode has one process-wide event bus that everything else in the UI listens to: `crates/jcode-base/src/bus.rs`.

```
// jcode: crates/jcode-base/src/bus.rs:499-502
pub fn global() -> &'static Bus {
    static INSTANCE: OnceLock<Bus> = OnceLock::new();
    INSTANCE.get_or_init(Bus::new)
}
```

A lazily-initialized, process-wide singleton. Structurally, `Bus` is just a `broadcast::Sender<BusEvent>` (plus some private debounce bookkeeping) — initialized with a fixed 256-slot ring buffer (`broadcast::channel(256)`). `BusEvent` is a roughly-30-variant enum; the ones you'll recognize from Chapter 3 are `BatchProgress` (the batch tool's incremental progress), `FileTouch`, `SwarmOutputTail`, and `SwarmAwaitCompleted` — the exact event `finalize_await` publishes when a background wait completes, which is the hand-off point between Chapter 3's Pattern 3 and this bus.

Publishing is deliberately unconcerned with whether anyone's listening:

```
// jcode: crates/jcode-base/src/bus.rs:525-533 (publish, abbreviated)
let _ = self.sender.send(event);
```

The send's `Result` is thrown away — zero receivers isn't a failure for a fire-and-publish bus. There's one carve-out: `UpdateStatus` events are also cached in a separate mutex, specifically so a subscriber that connects *after* the last publish can still ask for the latest value (`Bus::latest_update_status()`) instead of missing it entirely — a documented workaround for the fact that a broadcast channel has no general replay/history mechanism.

That last point matters for Chapter 3 readers: the fixed 256-slot buffer is exactly why `comm_wait.rs`'s handling of `RecvError::Lagged` matters in practice — any subscriber that falls more than 256 unconsumed events behind starts missing events, and each subscriber lags independently (a slow subscriber doesn't block the publisher or anyone else).

Worth naming explicitly, because it's a real topology distinction: the global `Bus` is *true* broadcast — one `tokio::sync::broadcast` channel, every subscriber gets every message. Swarm status/plan fan-out (`broadcast_swarm_status`, `broadcast_swarm_plan_with_previous`, `swarm.rs:685–970`) looks like broadcast from a distance — many recipients get the same update — but under the hood it's a loop over each member's own individual `mpsc` sender:

```
// looks like: fanout_session_event(swarm_members, &sid, event.clone())  
// called once per affected session id, not one send on a shared channel
```

So: one process has (at least) two different things that both produce a "many people got the same message" effect, but they're built on physically different primitives — a single shared broadcast channel vs. a fan-out loop over point-to-point channels. The TS analogy: the global bus is one `EventEmitter` topic everyone subscribes to; the swarm status/plan fan-out is a `Map<sessionId, sendFn>` you iterate and call individually.

The TypeScript equivalent

```
// idiomatic TS equivalent – this code does not exist in jcode
type BusEvent =
  | { type: "BatchProgress"; completed: number; total: number }
  | { type: "SwarmAwaitCompleted"; swarmId: string }
  | { type: "UpdateStatus"; status: string }; // ...and more variants

class Bus extends EventEmitter {
  private static instance: Bus;
  private lastUpdateStatus: BusEvent | undefined;

  static global(): Bus {
    return (Bus.instance ??= new Bus());
  }

  publish(event: BusEvent): void {
    if (event.type === "UpdateStatus") {
      this.lastUpdateStatus = event; // replay workaround, same as jcode
    }
    this.emit("event", event); // no error if nobody's listening
  }

  latestUpdateStatus(): BusEvent | undefined {
    return this.lastUpdateStatus;
  }
}
```

Node's EventEmitter doesn't drop events under backpressure the way a bounded broadcast channel does — it just keeps emitting synchronously to whoever's attached. The translation above keeps the API shape (global singleton, fire-and-forget publish, one cached "last value" escape hatch for late subscribers) without claiming the same backpressure behavior, since that would be dishonest about how EventEmitter actually works.

The interesting finding: SwarmRole/SwarmLifecycleStatus and the rewrite-on-reload

Here's a genuinely worthwhile finding, not just a code tour. You may have noticed above that all the live routing and permission checks compare role/status as bare strings — `member.role == "coordinator"`, `member.status == "running"`. That's because, in memory, they *are* plain strings:

```
// jcode: crates/jcode-app-core/src/server/state.rs:204,218 (SwarmMember
    fields)
status: String, // "Lifecycle status (ready, running, completed, failed,
    stopped, etc.)"
role: String,   // "Role: \"agent\" or \"coordinator\""

```

But there's a second, typed representation used at the persistence boundary — `SwarmMemberRecord`, in `crates/jcode-swarm-core/src/lib.rs`, with `status: SwarmLifecycleStatus` (13 named variants plus an `Other(String)` catch-all) and `role: SwarmRole` (`Agent/Coordinator/Other(String)`). Its doc comment calls it "the durable, persistable portion of a swarm member."

So there are two vocabularies. Does anything actually convert between them, or do they just coexist? Conversion code exists, and it turns out to do more than a naive round-trip.

Saving: `SwarmMember::durable_record`, `state.rs:243–258`:

```
// jcode: crates/jcode-app-core/src/server/state.rs:243–258
pub fn durable_record(&self) -> SwarmMemberRecord {
    SwarmMemberRecord {
        session_id: self.session_id.clone(),
        working_dir: self.working_dir.clone(),
        swarm_id: self.swarm_id.clone(),
        swarm_enabled: self.swarm_enabled,
        status: SwarmLifecycleStatus::from(self.status.clone()),
        detail: self.detail.clone(),
        task_label: self.task_label.clone(),
        friendly_name: self.friendly_name.clone(),
        report_back_to_session_id:
            self.report_back_to_session_id.clone(),
        latest_completion_report: self.latest_completion_report.clone(),
        role: SwarmRole::from(self.role.clone()),
        is_headless: self.is_headless,
    }
}

```

`SwarmLifecycleStatus::from(String)` and `SwarmRole::from(String)` (`jcode-swarm-core/src/lib.rs:174–193` and `:107–115`) match the string against every known literal — `"ready" => Self::Ready`, `"running" => Self::Running`, and so on — and fall through to `Self::Other(value)` for anything unrecognized. The conversion is total (it never panics) and open-world (an unrecognized string is preserved rather than dropped), but that also means a typo in a status string silently becomes `Other("...")` rather than getting caught as a bug.

The reverse direction, loading, calls `.as_str()` on each enum to get back a canonical lowercase string. For any string that maps to a *named* variant, the round trip is lossless — but only if the input matched the canonical lowercase form to begin with. "Ready" (capital R) would land in `Other("Ready")` on the way in, not `Ready` — a real, if narrow, footgun that the typed round-trip doesn't fully protect against.

That much would already be a reasonable chapter finding: a hand-written, total, open-world string↔enum conversion, called on every swarm-state save/load cycle (confirmed via `swarm_persistence.rs`'s `to_persisted_member/from_persisted_member`, which call `durable_record/from_record` respectively). But there's a sharper detail underneath it.

The round trip is not actually an identity function in practice, because of what happens on load, in between reading the persisted record and reconstructing the live `SwarmMember`:

```
// jcode: crates/jcode-app-core/src/server/swarm_persistence.rs:341-390
// (recover_member_status, paraphrased structure – see file for exact
//    match arms)
```

`recover_member_status` deliberately rewrites certain statuses as they come back off disk, because a live process context — the running task, the open event channel — cannot survive a server restart even though the *status string* that was written to disk can:

- A persisted `Running` status becomes `Crashed` on load — nothing can genuinely still be "running" after the server that was running it restarted.
- A persisted `Ready` status becomes `Stopped` on load. The code's own comment explains the bug this fixes: without this rewrite, every reload would resurrect hundreds of detached historical clients as "ready" again.
- A headless member sitting in any non-terminal status becomes `Crashed` too, with an explicit carve-out for statuses that are legitimately final (`Completed`, `Done`, `Failed`, `Stopped` are left alone).

So the answer to "does a translation layer exist between the two vocabularies" isn't just yes — it's that the translation layer is *where the codebase enforces that a persisted status can never lie about having survived a restart*. That's the enum's real behavioral job in this system, beyond serialization:

recover_member_status pattern-matches on it, and because it's a compiler-checked enum rather than string literals, that match is exhaustive — you can't add a new SwarmLifecycleStatus variant and forget to decide what recover_member_status should do with it without the compiler telling you.

Read as a design lesson, this is a nice concrete instance of "put the type system where correctness actually matters, not uniformly everywhere." Live, in-process code compares against string literals freely, because those comparisons are cheap and the strings never leave the process boundary in a way that risks silent corruption. The one place that *does* risk something expensive and hard to notice — silently believing a session is still running when the process that ran it is gone — is exactly where the codebase opts into a typed, exhaustively-matched enum instead.

The TypeScript equivalent

```
// idiomatic TS equivalent – this code does not exist in jcode
type SwarmLifecycleStatus =
  | "ready" | "running" | "completed" | "failed" | "stopped"
  | { other: string };

function statusFromString(s: string): SwarmLifecycleStatus {
  switch (s) {
    case "ready": case "running": case "completed":
    case "failed": case "stopped":
      return s;
    default:
      return { other: s }; // open-world: preserved, not dropped
  }
}

// The interesting part: rewriting on load, not a pure round-trip.
function recoverMemberStatus(
  persisted: SwarmLifecycleStatus,
  isHeadless: boolean,
): SwarmLifecycleStatus {
  if (persisted === "running") return "failed"; // "crashed" – nothing
  // survives a restart
  if (persisted === "ready") return "stopped"; // don't resurrect stale
  // ready clients
  if (isHeadless && ![ "completed", "failed",
    "stopped" ].includes(persisted as string)) {
    return "failed";
  }
  return persisted;
}
```


The lesson to take from this pattern, independent of language: a persistence boundary is a good place to ask "what part of this state description was only ever true because a process was alive?" — and to rewrite it explicitly on load rather than trusting it round-trips unchanged.

What this chapter left open

In the interest of not smoothing over gaps: the soft-interrupt injection mechanics behind "queued and injected at safe points" were not independently verified against `queue_soft_interrupt_for_session`'s implementation — that claim is sourced from the design doc, not from reading the injection code itself. Similarly, the swarm-specific `event_history: Arc<RwLock<VecDeque<SwarmEvent>>>` mentioned earlier as a contrast to the global bus's lack of replay is cited from Chapter 4's territory, not independently re-opened here. And this chapter deliberately didn't try to reproduce the literal tool-call parameter shape a caller uses to trigger a DM vs. broadcast vs. status read — that's a schema-level detail outside what was verified for this pass.

Chapter 8 — Design-Pattern Glossary & Further Reading

You've now seen jcode's swarm machinery from several angles: spawning, fan-out/fan-in, cancellation, shared state, the task DAG, communication. This chapter is the cheat sheet — pattern name, where it lives, the Rust primitive, and the TS primitive you'd reach for instead. Use it as an index back into the codebase, not a substitute for the chapters.

Two notes before you scan it. Not every pattern has a clean TS equivalent — a few exist specifically because Rust gives you real OS-thread parallelism and single-threaded TS doesn't, and the table says so rather than forcing a false match. And `dag::Mode` (deep/light task-graph rigor, Chapter 6) and `SwarmSpawnMode` (visible/headless/inline/auto window rendering, row 15) are two unrelated concepts that happen to share a name — don't conflate them.

Fan-out / fan-in combinators

Pattern	jcode file:line	Rust primitive	TS equivalent
Planner → fan-out → fan-in → integrate ¹	swarm.rs:1615– 1701, fan-in at 1671	try_join_all — all-or-nothing, ordered Vec<T>	Promise.all(tasks) — near-exact match
Concurrent tool calls, incremental progress	tool/batch.rs:282– 295, drain 300–317	FuturesUnordered — yields completion order, not submission order; re-sorted after	No exact match; closest is an async generator yielding as tasks settle. Contrast Promise.all, which yields once, at the end
Event-driven fan-in via broadcast + select	comm_await.rs:209– 303, select! 271– 300	broadcast::Receiver::recv() — can return Lagged(n) if a slow receiver falls behind the ring buffer	No built-in match for "lossy pub/sub with an explicit lag signal"; closest is an EventEmitter whose slow consumer silently drops events — Rust tells you
any/all wait mode, live- updating not one- shot	comm_await.rs:95– 126	Iterator::any/all re-evaluated inside a polling loop	Targets Promise.race/all, but as a repeatedly-checked predicate over shared state, not a one-shot combinator

¹ Debug-tooling path only (debug_command_exec.rs:132–139, debug_jobs.rs:77–95) — not reachable via the live swarm tool; see Chapter 2/3.

Cancellation & lifecycle

Pattern	jcode file:line	Rust primitive	TS equivalent
Fire-and-forget background task	comm_session.rs:772–822	tokio::spawn, JoinHandle dropped, never awaited	An async function called without await — a "floating promise"; errors only surface via a side effect
Structured concurrency scope, deadlock-safe shutdown	RuntimeTaskScope, runtime.rs:33–79	JoinSet<> + CancellationToken (hierarchical via child_token()); shutdown drains the JoinSet from under its Mutex before awaiting, avoiding a lock-across-await deadlock	AbortController threaded into every spawned op, plus a Set<Promise> awaited via Promise.allSettled after abort()
Cooperative, not preemptive, cancellation	runtime.rs:164–167/376–379/432–435	select! racing work against cancellation.cancelled(); nothing preempts a running task	Exactly AbortSignal semantics: code must check signal.aborted or use an abortable API
Async-aware interrupt flag, lost-wakeup-safe	InterruptSignal, jcode-agent–runtime/src/lib.rs:32–117	AtomicBool + Notify (async condvar) + an AtomicU64 epoch so a racing fire() is never lost	Mostly moot — single-threaded JS has no gap between "check aborted" and "register a listener." A "Rust is genuinely harder here" callout, not a port target

Shared state & singletons

Pattern	jcode file:line	Rust primitive	TS equivalent
Arc<RwLock<HashMap<...>>> registries	state.rs:108–113, runtime.rs:90–120 (double-locking: registry under RwLock, each Agent under its own Mutex)	Arc (refcount) + RwLock (many readers OR one writer)	No real equivalent in single-threaded JS. Either careful await placement replaces locking, or genuine worker_threads with message-passing to one owning thread (more idiomatic than SharedArrayBuffer)
Process-wide lazy singleton behind a lock	Bus::global(), bus.rs:499–502; also swarm.rs:108–113, state.rs:15–38	OnceLock/LazyLock + get_or_init	let instance; export function getBus() { if (!instance) instance = new Bus(); return instance; } — Node's module cache gives "once" almost free
Debounce instead of locking harder	swarm.rs:742–812, bus.rs:542–606 (750ms window)	State (last_published_at) checked before every publish	A debounce/coalesce wrapper (lodash.debounce or hand-rolled setTimeout) around .emit()

Pub/sub & event delivery

Pattern	jcode file:line	Rust primitive	TS equivalent
Global broadcast bus, closed tagged event vocabulary	BusEvent (~30 variants), bus.rs:394–466; channel(256) at 512	tokio::sync::broadcast — each subscriber gets its own cursor into one bounded ring buffer; slow subscribers lag independently	A process-wide EventEmitter with a closed, documented event-name set (ideally a discriminated union). No built-in bounded-buffer/lag signal — that part is genuinely tokio-specific
Targeted unicast-per-member fan-out (vs. the bus)	swarm.rs:836–920, delivery loop 899–908	Per-connection mpsc::UnboundedSender, fanned out manually	Iterating a Map<sessionId, WebSocket> and .send()-ing to each match — targeted, not a shared topic
Replay buffer beside a broadcast channel that has none	event_history, runtime.rs:107, populated by swarm.rs:1233–1257	A bounded VecDeque behind a lock, appended alongside every broadcast send	A ring-buffer array (push/shift at length N) kept beside an EventEmitter
Bidirectional two-map-mirror index	ChannelIndex, jcode–swarm–core/src/lib.rs:235–340	Two nested HashMaps kept in lockstep for near-O(1) lookups both directions	Map<string, Map<string, Set<string>>> pair, same dual-index, same manual-sync discipline

Indexing & authorization

Pattern	jcode file:line	Rust primitive	TS equivalent
Spawn ancestry via parent-pointer walk (no stored tree)	swarm.rs:41-60	Iterative lookup over <code>HashMap<String, SwarmMember></code> , cycle-guarded with a visited set, following <code>report_back_to_session_id</code>	A <code>Map<string, Member></code> plus a while loop up a <code>reportsTo</code> field, with a <code>Set<string></code> visited-guard
Authorization via ancestry (subtree ownership)	swarm.rs:76-85	Boolean check reusing the ancestry walk above	Same Map walk-up used as an auth predicate before a stop/control action
Mode-gated spawn rendering — not dag::Mode	<code>SwarmSpawnMode</code> , jcode-config- types/src/lib.rs:646-657	<code>#[derive(Default)] enum: Visible/Headless/Inline (default)/Auto</code>	A string-literal union with a default parameter value

API / argument shape

Pattern	jcode file:line	Rust primitive	TS equivalent
Layered optional-parameter delegation chain	swarm.rs:1291–1527ish, each # [expect(clippy::too_many_arguments)] documenting the tradeoff	Layered free functions, each widening the parameter list by one field	An options-object pattern (opts?: { report?; tldr? }) — TS sidesteps this for language reasons, not because the Rust is wrong
Structured, actionable validation errors	DagError + Display, dag/mod.rs:442–531 (11 variants)	An error whose message tells the caller the exact next step	A discriminated-union Result error whose formatter gives the same actionable text, not a bare "validation failed"

The task DAG's own idioms (Chapter 6)

Pattern	jcode file:line	Rust primitive	TS equivalent
Validated core wrapped by a stateful adapter ("lift, apply, lower")	jcode-plan/src/bridge.rs:1–9, to_task_graph/apply_task_graph at 94/124	Free functions lifting a live VersionedPlan into a pure TaskGraph, applying an op, lowering back	liftToGr — arguably a lesson here
Stage-on-a-clone, validate, commit-or-reject	dag/ops.rs: expand_node (227–367, clone 287, commit 365), inject_from_gate (444–540)	Clone the graph, mutate the clone, run cycle_nodes(), only reassign if valid	Build a new validate if "real" state
Encapsulation via a private field + validated methods	TaskGraph.nodes: Vec<TaskNode> private, dag/mod.rs:539–542	No pub on the field — mutation forced through ops::	A class with getter/m
Feature flag reduced to one predicate function	dag::Mode/requires_gates(), dag/mod.rs:37–49	Consulted inside expand_node/complete_node, never scattered as ad hoc checks	A single boolean
Lenient free-text-to-enum parsing, substring-order traps	ConfidenceLevel::parse, dag/mod.rs:141–204	Negation phrases checked before word-rung substrings, so "not confident" can't match "confident" → High	parseConf same neg
Word-boundary-safe substring matching	mentions_node_id, ops.rs:593–628	A hand-rolled boundary scanner treating <code>_.:</code> as legal id characters	A RegExp scanner with regex <code>\b</code>
Side-map extension instead of widening the base record	NodeMeta, jcode-plan/src/lib.rs:117–146, keyed in VersionedPlan.node_meta	A companion map keyed by id, instead of optional fields only one subsystem needs	Map<Task> beside Task

The agent-facing protocol surface

Pattern	jcode file:line	Rust primitive	TS equivalent
Prompt text as the LLM-facing protocol, separate from the Rust API	jcode-swarm-core/src/lib.rs:390-433, 449-499	Functions generating <code><system-reminder></code> text naming the only legal turn-ending actions	No real TS "pattern" — the tool schema is the enforced contract, but the natural-language directive is what gets the model to use it correctly; the two must be kept in sync by hand
The verified tool-call chain: prompt → schema → match arm → wire request → handler → engine	schema at tool/communicate.rs:1959 → match arm 2629-2656 → server/comm_graph.rs:339-402 → dag::expand_node at line 368	A typed request enum dispatched through a server handler into a pure engine function	A typed RPC dispatcher (<code>Record<ActionName, (params) => Promise<Result>></code>) whose handlers call pure domain functions

One caveat: the exact wire-JSON shape of `Request::CommExpandNode` and friends, as serialized in `jcode-protocol`, wasn't independently opened during this handbook's research. The Rust-side chain above — prompt text through the `dag::` function call — is fully verified; the literal wire format is an open question if you go looking.

Further reading

- `docs/SWARM_ARCHITECTURE.md` (318 lines) — the older, agent-first design doc, explicitly "largely implemented" but superseded in framing by the task-graph doc below. Best source for vocabulary and lifecycle: roles and mode-gated spawning (22-58), lifecycle states and notifications (89-106), completion report policy (108-124), communication (194-254), and the "no locks" language (307-311) that Chapter 5 adds nuance to. One flag: its 8-state lifecycle list doesn't map 1:1 to the code's actual `SwarmLifecycleStatus` enum (13 variants plus `Other`, `jcode-swarm-core/src/lib.rs:136-151`) — read the doc as intent, the enum as current ground truth.

- **docs/SWARM_TASK_GRAPH.md** (605 lines) — the newer, DAG-first doc Chapter 6 draws from most heavily. Worth reading directly: §1a's Deep vs. Light table (54-98), §5's dataflow description (195-216), §6.4's "implemented enforcement" section (275-304, the richest passage tying prose to real DagError variants), §8a's communication-rework notes (407-475), and §9, the worked example Chapter 6 walks through in full (478-549).
- **crates/jcode-plan/src/dag/** — the DAG engine: `mod.rs` (types), `ops.rs` (validated mutations), `schedule.rs` (ready-set, dispatch, dataflow hydration), `tests.rs` (1392 lines — the best source for every edge case the DagError variants exist to catch). `sim.rs` (155 lines, a deterministic simulator) wasn't opened during this handbook's research — a genuine go-read-it-yourself pointer.
- **crates/jcode-app-core/src/server/** — `swarm.rs` (member registry, plan-driven fan-out), `comm_session.rs` (spawn path), `comm_await.rs` (fan-in wait), `comm_graph.rs` (DAG tool-call handlers), `state.rs` (SwarmState/SwarmMember).
- **crates/jcode-app-core/src/tool/batch.rs** and **communicate.rs** — the intra-agent batch tool and the swarm tool's full action surface (roughly 30 actions; this handbook covers a handful).
- **crates/jcode-base/src/bus.rs** — the global event bus and its full BusEvent vocabulary (~30 variants, 394-466), most of which this handbook doesn't individually cover.
- **crates/jcode-agent-runtime/src/lib.rs** — `InterruptSignal` and its five regression tests (142-283), pinning down the lost-wakeup race from Chapter 4.

Everything in this handbook cites back to one of these files or docs — if a claim here doesn't line up with what you find when you open them, the code is the ground truth, not the prose.